



INDIAN INSTITUTE OF TECHNOLOGY MADRAS

DEPARTMENT OF ELECTRICAL ENGINEERING

MASTER'S THESIS

ACCELERATING DEEP LEARNING USING
PHOTONIC PROCESSORS

AUTHOR

ANIKET KUKRETI

Roll No. EP19B036

SUPERVISOR

PROFESSOR ANIL PRABHAKAR

JUNE 2024

ACKNOWLEDGEMENT

I would like to extend my sincere thanks to Professor **Anil Prabhakar** for his ongoing support and expertise in my project. His invaluable advice and guidance have been crucial, especially in the face of hardship. His patience with my learning process has been particularly appreciated as I navigated this challenging but enriching project.

I am thankful to my **family** and **friends** for their unwavering support and encouragement during this journey. Their belief in me and their support have been a constant source of motivation and strength throughout the course of this project.

Lastly, I would like to express my appreciation to **IIT Madras** and the support staff for providing the necessary resources, environment, and assistance for my research. The facilities and guidance offered by the university have been greatly supportive of my academic efforts.

ABSTRACT

As the computational demands of state-of-the-art artificial intelligence (AI) algorithms continue to outpace the growth of traditional hardware capacities, optical computing has emerged as an indisputable alternative to traditional compute. This thesis focuses on Optical Neural Networks (ONNs), specifically focusing on integrated photonic architectures, which offer notable benefits in terms of low latency and scalability. A significant challenge associated with these architectures is the inherent fabrication errors and imprecisions, highlighting the necessity for precisely calibrating such devices.

The core of this research is centred around the characterization and calibration of a specifically designed and packaged 6×6 Mach-Zehnder Interferometer (MZI) mesh chip. The efficacy of ONNs is gauged by simulating and training ideal ONNs for machine learning tasks, thereby establishing performance benchmarks. An analysis of the mechanisms behind the chip was done to facilitate the development of calibration algorithms. Global optimization techniques were assessed for their viability in estimating component-level phase errors, and a novel algorithm named Unitary Isolation was introduced to calibrate the mesh incrementally. The findings and methodologies presented in this work serve as a stepping stone for further advancements in optical computing devices.

Keywords: optical neural networks, optical computing, mach-zehnder interferometer, calibration

CONTENTS

Contents	iii
1 Introduction	2
1.1 Background and Motivation	2
1.2 Research Objectives	3
2 Brief Review of Literature	4
2.1 Optical Neural Networks	4
2.2 Mesh Architectures	5
2.3 Mesh Calibration	7
3 Theoretical Background	10
3.1 Unitary Mesh	10
3.2 SVD Mesh	12
3.3 Symmetric MZI Modification	13
4 Methods	17
4.1 Mesh Calibration	17
4.1.1 Optimisation Approach	17
4.1.2 Optimisation Methods	19
4.1.3 Unitary Isolation Algorithm	19
4.2 Training an Optical Neural Network	20
4.2.1 Preprocessing	21
4.2.2 Network Architecture	21
4.2.3 Multi-core ONNs	22
4.2.4 Robustness Analysis	23
5 Results	24
5.1 Phase Error Estimation	24
5.1.1 Gradient Descent	24
5.1.2 PSO	30
5.1.3 Unitary Isolation Algorithm	31
5.2 Optical Neural Network	34

5.2.1 Classification Results 34

5.2.2 Robustness Analysis 42

6 Conclusion and Future Work 43

INTRODUCTION

1.1 Background and Motivation

The domain of optical computing has emerged as a beacon of innovation and possibility. Presently, state-of-the-art artificial intelligence algorithms are doubling in compute requirements every 3 to 4 months, whereas hardware capacity is doubling every 18 months [1]. With a $5\times$ divergence in growth, there's a growing call for new ways to handle such information.

Optical Neural Networks (ONNs) have garnered quite the attention due to their advantageous characteristics, such as low latency, extensive parallelization capabilities and minimal power consumption [2]. These attributes make it a promising candidate for accelerating deep learning computations. There are several architectures out there, based on either free space interconnections or waveguide interconnections [3]. In this study, the architecture of choice is waveguide based, mainly motivated by the desire to implement a neural network on chip TO leverage the capabilities and scalability of integrated photonic technology.

Advances in photonic integrated circuits have recently enabled electrically reconfigurable optical systems that can implement universal linear optics transformations. Mach Zehnder Interferometer (MZI)-based linear optical processors have gained prominence as a versatile tool in both classical and quantum photonics. These devices are typically constructed using a mesh of beam splitters and phase shifters, which allows for straightforward fabrication via integrated photonic fabs and ensures scalability. The inherent properties and design of these photonic devices enables the application of arbitrary unitary transformations to a vector of input modes, thereby offering a promising hardware platform for fast and energy-efficient machine learning powered by light.

The operation of these interferometers relies on the precise implementation of the required phase shifts to realize the transformation matrix specific to the application. These phase shifts are theoretically determined to match the desired transformations. However, due to fabrication-induced waveguide geometry variations, along with

imperfect beamsplitter ratios, random phase offsets occur in the MZIs, which leads to erroneous outputs. Hence, there is a need for the configuration and calibration of such chips in order to make them feasible for experimental use.

1.2 Research Objectives

The overarching objective of this thesis is to explore and characterize a packaged 6×6 Mach-Zehnder Interferometer (MZI) mesh architecture chip. This research aims to implement and assess the performance of a neural network on this chip, with the following specific objectives:

1. **Simulation and Benchmarking:** Simulate ideal optical neural networks to ascertain the upper limits of performance on photonic chips. Assess the programmed photonic mesh's performance against conventional artificial neural networks (ANNs) in training and inferencing phases, highlighting the comparative advantages and limitations.
2. **Robustness Analysis:** Evaluate the impact of fabrication errors on the accuracy of the chip's output. This includes an assessment of sensitivity to component-level phase errors and their implications on network performance.
3. **Chip Calibration:** Develop and implement algorithms tailored for calibrating the 6×6 SVD mesh MZI chip. This involves devising experimental approaches to estimate the component-level phase errors within the chip, thereby enhancing its operational accuracy and reliability.

These objectives collectively aim to advance the capabilities of photonic computing by optimizing the functionality and efficiency of MZI mesh architectures, setting the stage for more sophisticated implementations in optical neural networks.

BRIEF REVIEW OF LITERATURE

2.1 Optical Neural Networks

Optical Neural Networks (ONNs) are neural networks implemented on optical processors, which can be either digital-optical hybrids [4, 5, 6] or fully optical [7, 8, 9] setups. The implementation of ONNs leverages the unique properties of light to perform neural network operations, particularly matrix multiplication, with high speed and accuracy.

Neural networks consist of interconnected neurons with associated weights. The forward and backward propagation of signals through these networks can be modeled using matrix multiplication, a fundamental operation whose efficiency is crucial for the performance of neural networks. In ONNs, these linear operations are carried out using optical hardware that exploits the physics of electromagnetic waves to achieve light-speed computations. The input to the network is encoded into light beams, the weight matrices are programmed into the optical processing units, and the output is detected using photodetectors.

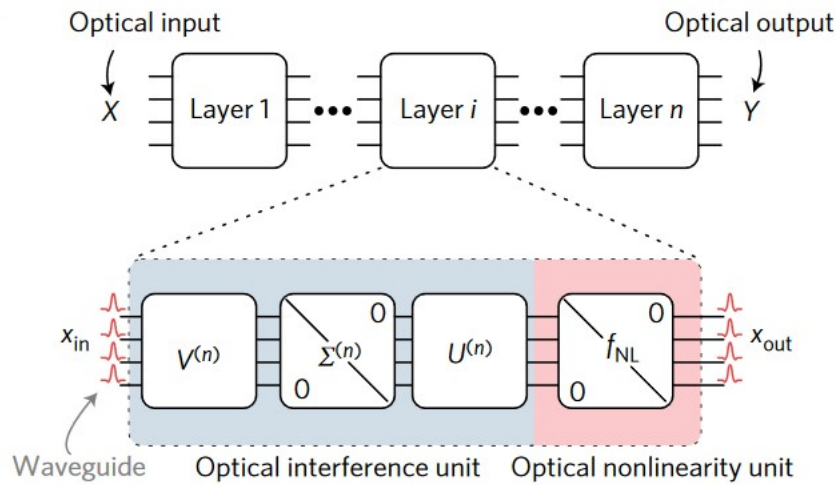


Figure 2.1: Structure of Optical Neural Networks

There are three major methods of photonic matrix-vector multiplication [10]:

1. **Plane Light Conversion (PLC):** Utilizes light diffraction in free space to implement matrix multiplication.
2. **Mach-Zehnder Interferometer (MZI):** Based on factorization of unitary matrices into a sequence of MZI transformations. Typically done for coherent light.
3. **Wavelength Division Multiplexing (WDM):** Employs microring resonators to adjust transmission coefficients, facilitating incoherent matrix computation.

As our focus is on integrated photonics applications, both the MZI and WDM methods are practical options. MZI systems are effective due to their single laser source requirement, ease of reconfigurability, and high precision. However, they suffer from a lack of parallelism and scalability limitations. In contrast, WDM approaches offer improved scalability and inherent parallelism, but they necessitate intricate control circuits for stability due to the use of MRRs [11]. Conversely, MZIs do not require such complex stabilization, making them more straightforward and potentially more reliable for large-scale deployments.

There have been numerous experimental realisations of MZI-based ONNs that have been used for machine learning tasks, including spoken vowel recognition [7, 9] and MNIST dataset classification [12]. These studies have consistently achieved accuracies above 90%, highlighting the practicality and competitive edge of these devices. A distinctive feature of ONNs is their ability to perform matrix-vector multiplications (MVM) between complex elements [4], assuming the presence of coherent detectors within the circuit. Implementing complex-valued neural networks on digital hardware typically demands more resources than their real-valued counterparts. Yet, MZI-based ONNs naturally support complex-valued neural network architectures, which further proves their scope and applicability.

2.2 Mesh Architectures

There are several mesh architectures for implementing unitary matrices, each with unique characteristics:

- **Reck Topology [13]:**
 - *Overview:* The Reck topology was the first coherent Mach-Zehnder Interferometer (MZI) mesh capable of factorizing an $N \times N$ unitary matrix using MZI transformations. This triangular mesh (Fig. 2.2a) implements a unitary matrix using $\frac{N(N-1)}{2}$ MZIs and is simple to calibrate.
 - *Advantages:* Simple to calibrate as MZIs can be independently calibrated.
 - *Disadvantages:* It is sensitive to insertion losses due to its high optical depth, non-compact design and asymmetric arrangement.
- **Clements Topology [14]:**
 - *Overview:* The Clements topology employs a rectangular mesh configuration

- (Fig. 2.2b) with the same number of MZIs but in a more compact arrangement.
- *Advantages*: Offers reduced loss and a more compact design compared to the Reck topology.
 - *Disadvantages*: Poses greater challenges in calibration [15] due to the more complex arrangement of the MZIs.
 - **Diamond Topology [16]**:
 - *Overview*: The Diamond topology (Fig. 2.2c) introduces a symmetrical design to mitigate some of the issues found in the Reck topology. This design incorporates additional MZIs, with a total of $(N - 1)^2$, to reduce phase errors and compensate for insertion losses.
 - *Advantages*: Provides enhanced phase control and flexibility in phase configuration. Simple to calibrate as MZIs can be independently calibrated.
 - *Disadvantages*: Still suffers from imbalanced insertion losses due to asymmetric arrangement and additional MZIs.
 - **Bokun Topology [15]**:
 - *Overview*: The Bokun topology (Fig. 2.2d) combines elements of both the Diamond and Clements topologies. Optimized for energy efficiency, it features a minimum optical path depth, enhancing scalability and resilience to loss and fabrication imperfections.
 - *Advantages*: Easier phase monitoring and programming, maintains high computational accuracy at high reprogramming speeds. Simple to calibrate as MZIs can be independently calibrated.
 - *Disadvantages*: Increased the size and energy consumption of the chip, which may not be desirable for all applications.

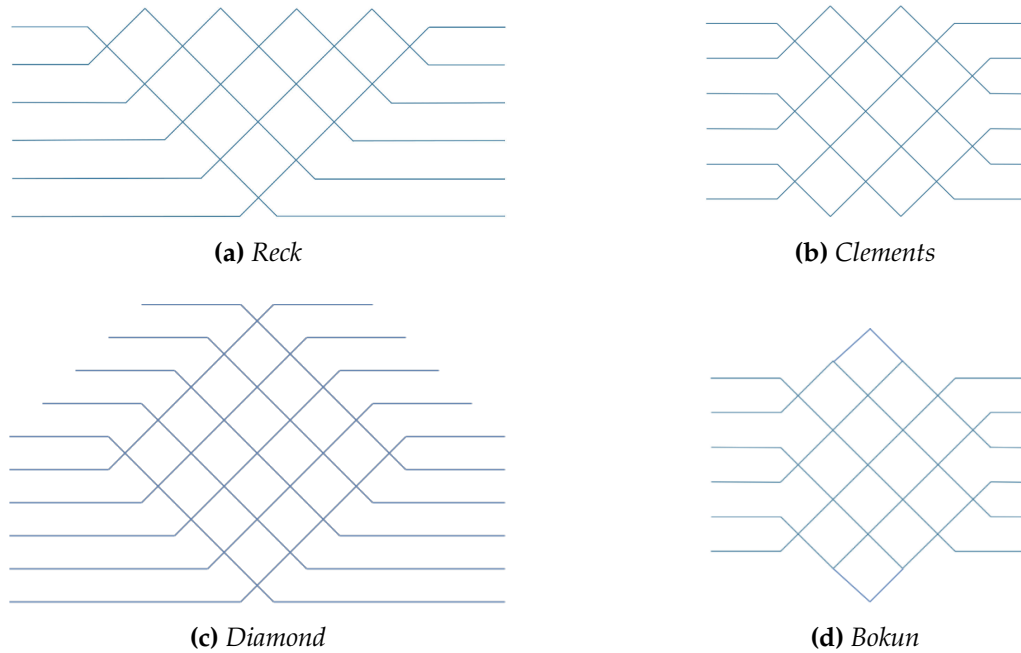


Figure 2.2: Different types of mesh topologies for 6×6 unitaries. Each crossing denotes an MZI.

The performance of these meshes in ONNs were tested in [15, 17] by classifying a linearly separable dataset. The results of these studies suggest that the Clements mesh is much more robust to phase uncertainty and optical loss when compared to Reck and Diamond (see Fig. 2.3). While the Bokun mesh gives similar performance to Clements, it increases the size and energy consumption of the chip, which may not be desirable based on our applications.

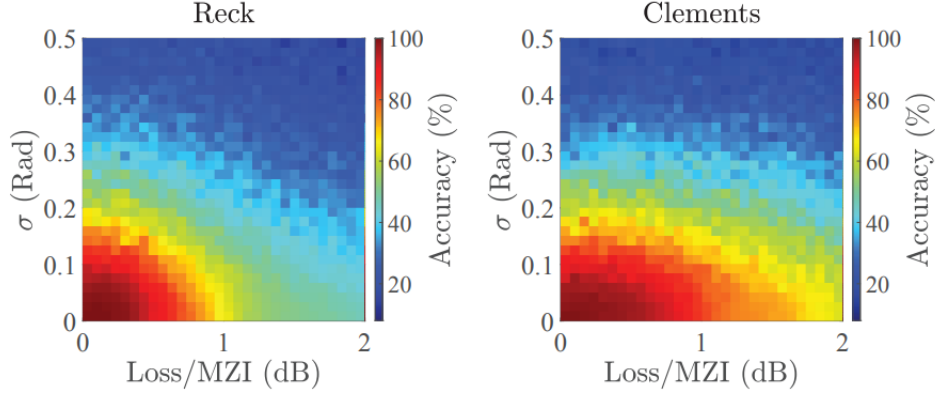


Figure 2.3: Comparison of 4×4 Reck and Clements mesh phase error tolerance when performing a classification task [17]

The aforementioned topologies can only simulate unitary matrices. However, in applications like deep learning, simulating arbitrary weight matrices is essential. Any arbitrary matrix M can be decomposed as,

$$M = U\Sigma V^\dagger$$

using Singular Value Decomposition (SVD), where U and V are unitary matrices and Σ is a diagonal matrix with non-negative real numbers. The Σ matrix can be implemented in integrated photonics using **attenuators** or **Mach-Zehnder Modulators (MZMs)**. Therefore, by leveraging the existing unitary architecture, an arbitrary matrix can be realized by concatenating the architectures for U , Σ , and $V^\dagger$.

The chip that was studied in this project is a 6×6 SVD mesh, with the unitary blocks utilising Clements topology. The Clements mesh was chosen taking into account all of the above tradeoffs. In the next section, the progress on chip calibration techniques will be discussed.

2.3 Mesh Calibration

Calibration of MZI meshes is critical for their operation, particularly when components are not ideal. The effectiveness and accuracy of MZI meshes rely on their ability to reproduce ideal transfer matrices and responses. Errors such as phase uncertainties, optical loss, beamsplitter inaccuracies, and cross-talk significantly impact performance. As discussed previously, selecting an appropriate mesh topology can reduce the effects of

phase uncertainties and optical losses. Despite this, individual MZIs require calibration to maintain output accuracy.

The goal of calibration is to facilitate precise programming of MZIs by compensating for errors in beamsplitter ratios and phase shifts. Local error correction within photonic circuits requires a detailed characterization of each phase shifter and beamsplitter. A one time calibration is necessary for all components, the results of which are stored for future look up. Subsequently, programming of any arbitrary function is done by first determining the settings for an idealized mesh and then translating these settings to the corresponding settings for an imperfect device. [18]

Miller et al. [19] initially proposed self-configuration techniques for these meshes, which include the use of double MZIs or tunable beamsplitters. Their method involves tuning the beamsplitter ratios to an ideal 50:50, followed by phase calibration. Unfortunately, our mesh does not incorporate tunable beamsplitters or double MZIs.

Some proposed methods involve the use of non-invasive photodetectors at the outputs of internal MZIs [20, 21]. These devices allow the monitoring of the output transmission of each MZI in response to applied voltage of phase shifters. However, the chip being studied in this project lacks such monitoring capabilities, and integrating these components would necessitate refabrication. Thus, calibration must rely solely on external measurements.

To achieve an error-free local calibration using only external detectors, the following conditions must be met [22]:

1. Only one input of the MZI under characterization should be illuminated, ensuring there is no interfering signal from other inputs.
2. When an MZI is characterized through a sequence of other blocks, one of the following conditions are necessary:
 - (a) One input of the previous or successive MZIs should be null
 - (b) The set of previous or successive MZIs should be precisely biased in their bar or cross states.

The Clements mesh cannot ensure these conditions in a straightforward manner. Even when condition 1 is met, condition 2b is difficult to ensure. A noteworthy contribution by Alexiev et al. [23] is the **Node Isolation Algorithm** for rectangular interferometers. This method ensures an MZI is “isolated” by allowing light entry only through one port (Condition 1) and ensuring that light is only detected from one output. The process involves isolating MZIs algorithmically, beginning with the longest diagonal before advancing to the top and bottom diagonals, and employs “Co-sinusoidal Proportional Calibration” for internal phase estimation. This calibration leverages power maximization techniques to accurately determine the internal phase shifts (θ).

In the above figure, diamond boxes are calibrated MZIs, circle boxes are MZIs under calibration. Fig. 2.4a shows the calibration of 3rd node along the main diagonal. The

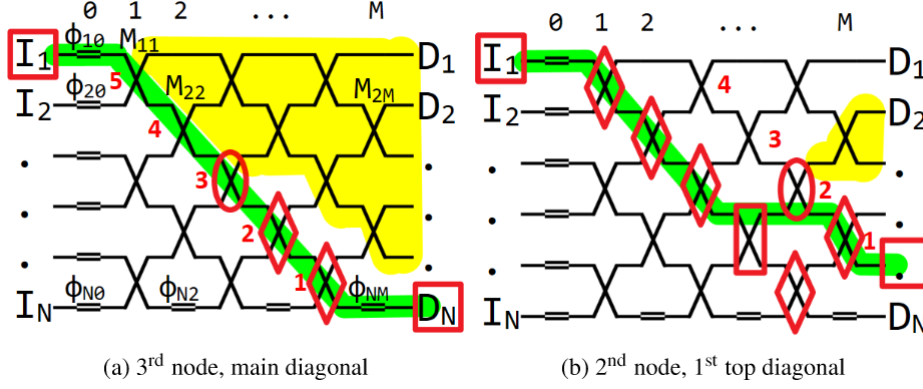


Figure 2.4: Node Isolation Algorithm steps detailed in [23]

main diagonal needs to be calibrated before any other diagonals in order to isolate the MZIs. Once that is done, we can move on to the calibration of top diagonal. Fig. 2.4b shows the calibration of 2nd node of the top diagonal.

Another promising calibration method tailored for Clements meshes was proposed by Hamerly et. al [24, 25]. This method requires only external photodetectors and manages to set the correct phase values in the presence of hardware errors. It reduces the effect of fabrication errors by a quadratic factor, making it a robust algorithm for setting phases. Its computational complexity is N^2 for on-chip operations and N^3 for operations performed on a computer, highlighting its efficacy as a calibration technique.

The methods detailed so far are only applicable for unitary meshes as they exploit the structure and mathematical principles inherent to these meshes. The only way to directly extend these methods to SVD meshes is to calibrate each unitary block individually, followed by with the Σ block. However, the SVD chip used in this project (see Fig. 3.3) does not provide direct access to the inputs and outputs of each unitary block. Hence, there's a serious need for calibration methods that can work on SVD meshes.

Global optimisation methods [26, 27] provide a way to tune meshes regardless of their specific structure. These methods treat the mesh as a black box, focusing solely on minimizing an objective function that quantifies the discrepancy between the desired and actual chip configurations. The drawback associated with these methods is their high computational cost, which scales with the number of optimization iterations. This makes the approach less suitable for applications requiring frequent mesh reconfigurations.

Therefore, accurate calibration of the chip necessitates a method that either leverages the chip's specific architecture or is agnostic to the mesh structure. Another avenue to explore is the isolation of the unitary blocks of our chip to enable the use of unitary mesh calibration methods. These ideas will be explore in the following chapters.

THEORETICAL BACKGROUND

3.1 Unitary Mesh

The MZI (Fig. 3.1) is the basic computing unit for all the mesh architectures, consisting of two couplers (beam splitters) and two phase shifters. The transformation matrix of an MZI is given by,

$$U_2(\theta, \phi) = L(\phi) \cdot B \cdot R(\theta) \cdot B = ie^{i\theta/2} \begin{bmatrix} e^{i\phi} \sin \frac{\theta}{2} & e^{i\phi} \cos \frac{\theta}{2} \\ \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \end{bmatrix} \quad (3.1)$$

where,

$$B = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & i \\ i & 1 \end{bmatrix}, \quad R(\theta) = \begin{bmatrix} e^{i\theta} & 0 \\ 0 & 1 \end{bmatrix}, \quad L(\phi) = \begin{bmatrix} e^{i\phi} & 0 \\ 0 & 1 \end{bmatrix}$$

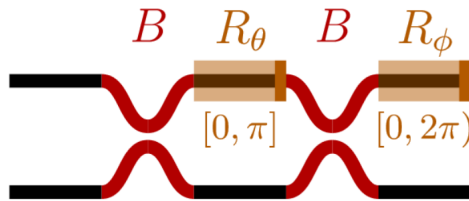


Figure 3.1: Architecture of an (asymmetric) MZI unit [26]

For a mesh network with N inputs, the effect of a single MZI connection (between the n and $n + 1$ arms) is represented by embedding the 2×2 unitary matrix (Equation 3.1) into an $N \times N$ identity matrix, changing only the elements around the n and $n + 1$ diagonal

entries. This is known as a **Givens rotation** [26], denoted by T_n .

$$T_n(\theta_{nl}, \phi_{nl}) := \begin{bmatrix} 1 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & & \vdots \\ 0 & \cdots & e^{i\phi_{nl}} \sin \frac{\theta_{nl}}{2} & e^{i\phi_{nl}} \cos \frac{\theta_{nl}}{2} & \cdots & 0 \\ 0 & \cdots & \cos \frac{\theta_{nl}}{2} & -\sin \frac{\theta_{nl}}{2} & \cdots & 0 \\ \vdots & & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & \cdots & 1 \end{bmatrix}$$

with non-trivial elements at the n and $n + 1$ rows and columns.

Any unitary matrix can be represented as a product of Givens rotations:

$$U = D(\gamma) \left(\prod_{(n,l) \in S} T_n(\theta_{nl}, \phi_{nl}) \right) \quad (3.2)$$

where S is an ordered sequence of specific rotations and $D(\gamma)$ is a diagonal matrix with modulus 1 entries, characterised by the $(\gamma_1, \dots, \gamma_N)$ parameters.

Since each MZI has two phase shifters, the total number of phases need to determine a unitary matrix is $N(N - 1) + N = N^2$. This agrees with the number of free components of a general unitary matrix, which is N^2 .

The **Clements Phase Decomposition algorithm** [14] is an algorithm that can determine the exact phases needed to implement a particular unitary in the Clements mesh. The algorithm, similar to Reck decomposition [13], uses Givens rotations to nullify the elements of a unitary matrix in a particular sequence that maps to the order of MZIs in the Clements architecture.

For example, right multiplying a Givens rotation, say $T_m(\theta, \phi)$, mixes columns m and $m + 1$ of the target unitary U . The element (j, m) can be nulled as follows,

$$(UT_m(\theta, \phi))_{j,m} = U_{j,m}T_{m,m} + U_{j,m+1}T_{m+1,m} := 0 \quad (3.3)$$

$$U_{j,m}e^{i\phi} \sin \frac{\theta}{2} + U_{j,m+1} \cos \frac{\theta}{2} = 0$$

$$\tan \frac{\theta}{2} e^{i\phi} = -\frac{U_{j,m+1}}{U_{j,m}}$$

$$\therefore \theta = 2 \arctan \left(\frac{|U_{j,m+1}|}{|U_{j,m}|} \right) \text{ and } \phi = \arg \left(-\frac{U_{j,m+1}}{U_{j,m}} \right) \quad (3.4)$$

Observe that the phases needed to null a particular element are uniquely determined up to π and 2π respectively. This suggests that the phases required to decompose a unitary might be unique. Upon nulling the elements of the auxillary unitary according to the sequence given in Fig. 3.2a, we are left with a diagonal unitary D . Inverting the Givens rotations (moving them to RHS) and shifting the matrix D to the end [14] yields Eq. 3.2.

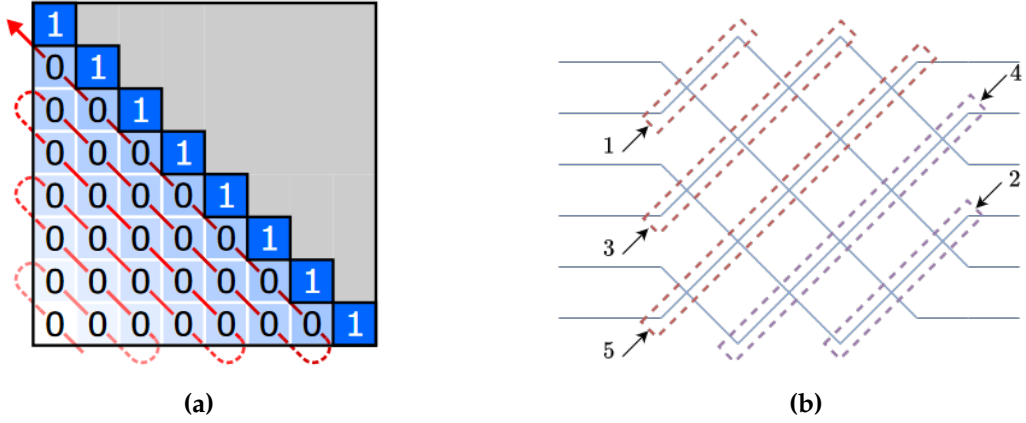


Figure 3.2: The order in which the elements of a unitary matrix are nullified in Clements Phase Decomposition [28]

3.2 SVD Mesh

Fig. 3.3 illustrates the architecture of the 6×6 packaged MZI chip, referred to as the **SVD mesh**. It consists of five functional blocks. The splitting tree takes a single input mode and splits it into six modes, with amplitudes controlled by attenuators in each branch. The U and V^+ blocks are 6×6 Clements meshes. The Σ block, composed of attenuators, scales the inputs, effectively acting as a diagonal matrix.

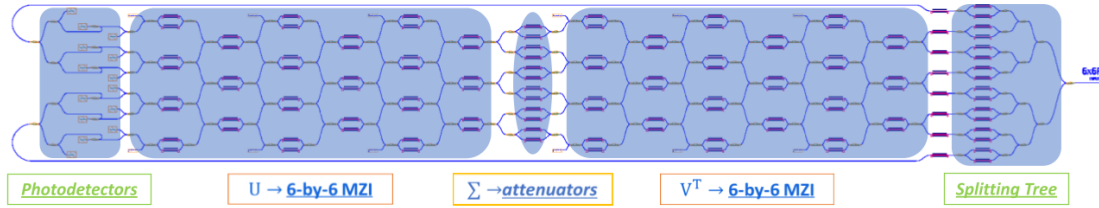


Figure 3.3: 6×6 SVD Chip Architecture

An attenuator or MZM is equivalent to an MZI with the bottom input and output arms blocked out. A close up of the attenuators present in the SVD chip is given in Fig. 3.4. The transmission function of this component is given by,

$$M(\theta_1, \theta_2) = e^{i(\theta_1 + \theta_2)/2} \cos\left(\frac{\theta_1 - \theta_2}{2}\right) \quad (3.5)$$

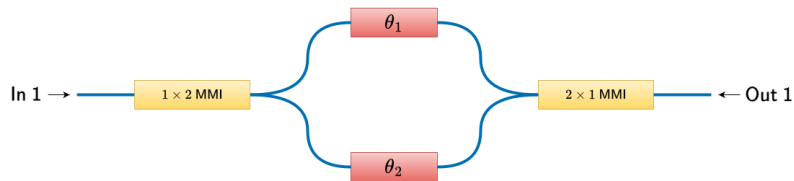


Figure 3.4: Architecture of an attenuator/MZM unit

After passing through these blocks, the transformed signals are recorded by photodetectors (PDs). Since PDs only return the signal intensity / power, the information encoded in the phase of the output signals would be lost. To address this, two reference arms extending to the left end of the chip are used. These reference signals are combined with the each of the transformed signals to perform coherent detection. Focusing on one reference branch and one output arm, we can redraw the PD block as showing in Fig. 3.5.

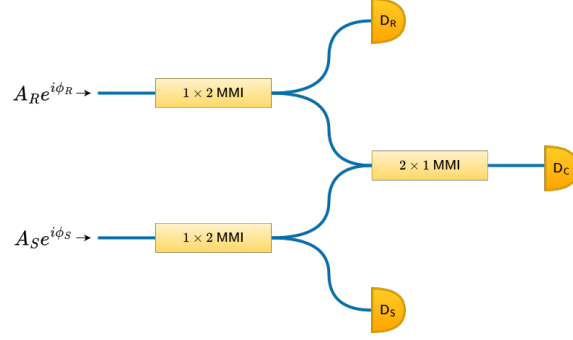


Figure 3.5: A simplified representation of the coherent detection scheme on chip

In the above figure, let S denote the signal under consideration and R denote the reference. In order to determine ϕ_S , the signals are put through splitters so that A_S and A_R can be estimated. The readings at D_S and D_R are $\frac{A_S^2}{2}$ and $\frac{A_R^2}{2}$ respectively. The other arms are combined using a 2×1 MMI and finally detected at D_C . Since the combined field is given by $\frac{1}{\sqrt{2}}(A_S e^{i\phi_S} + A_R e^{i\phi_R})$, the power detected would be,

$$PD_C = \frac{A_S^2}{2} + \frac{A_R^2}{2} + A_S A_R \cos(\phi_S - \phi_R)$$

The $\frac{A_S^2}{2}$ and $\frac{A_R^2}{2}$ terms can be subtracted as they've been measured. ϕ_R can be set to 0 and $\frac{\pi}{2}$, which would yield $A_S A_R \cos \phi_S$ and $A_S A_R \sin \phi_S$ respectively. Hence, taking the ratio of these two quantities enables the measurement of phase.

Note that the SVD mesh implements a complex-valued matrix, and performs MVM between a complex vector and complex matrix. The coherent photodetectors enable the capture of complex outputs. Hence, this chip can enable the implementation of complex-valued neural networks.

3.3 Symmetric MZI Modification

The configuration of the Mach-Zehnder Interferometers (MZIs) in the SVD mesh, as illustrated in Fig. 3.3, differs from the design depicted in Fig. 3.1. In the SVD mesh, both the phase shifters are placed in the internal arms of an MZI. This configuration of an MZI is called the **symmetric MZI** (sMZI), in contrast to the earlier shown **asymmetric MZI** (aMZI). Fig. 3.6 shows a close up of an sMZI.

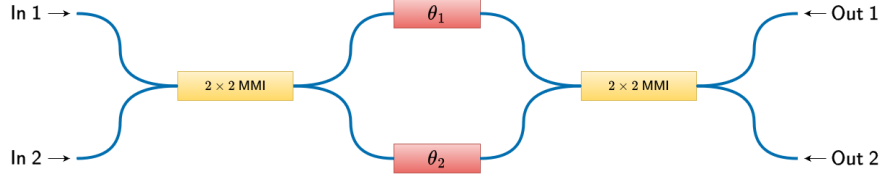


Figure 3.6: Architecture of a symmetric MZI unit

$$U_2(\theta_1, \theta_2) = ie^{i\frac{\theta_1+\theta_2}{2}} \begin{bmatrix} \sin \frac{\theta_1-\theta_2}{2} & \cos \frac{\theta_1-\theta_2}{2} \\ \cos \frac{\theta_1-\theta_2}{2} & -\sin \frac{\theta_1-\theta_2}{2} \end{bmatrix} \quad (3.6)$$

$$= ie^{i\omega} \begin{bmatrix} \sin \delta & \cos \delta \\ \cos \delta & -\sin \delta \end{bmatrix} \quad (3.7)$$

Setting $\omega = \frac{\theta_1+\theta_2}{2}$ and $\delta = \frac{\theta_1-\theta_2}{2}$ leads to a cleaner representation of the matrix and highlights the key difference between sMZIs and aMZIs. The sMZI cannot produce relative phase shift between its outputs, which makes it a non-universal 2×2 unitary transformation. It is not immediately clear whether such non-universal components can be combined to create a higher mode universal device. However, Bell et. al [29] provided a proof for the equivalence of sMZI and aMZI usage in Reck and Clements topologies. There are modifications that need to be made to these topologies in order to enable universal programming, but these changes are minimal.

The lack of a relative phase term in the transformation matrix of sMZIs causes Clements decomposition to fail. Initiating the procedure, like it was done in Eq. 3.3, is not possible as $\arg(U_{j,m+1}) \neq \arg(U_{j,m})$. This can be resolved by adding an external phase shifter at the input/output in order to match the phases of the two matrix elements. Once the decomposition process is started along a diagonal (Fig. 3.2b) and the first element is nulled, the global ω phase of the prior MZI can be used to equalise the arguments of the next matrix elements. The modified Clements mesh is shown in Fig. 3.7.

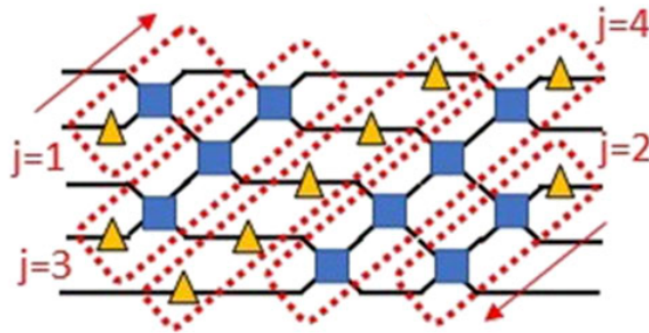


Figure 3.7: Modified 5×5 Clements mesh with sMZIs and added phase shifters [29]

The intermediate phase shifters can be further relocated to the empty edges of the mesh by exploiting the nature of sMZIs. Fig. 3.8 visualizes the idea behind relocation. Any internal phase between sMZIs can be absorbed into a prior or successive sMZI due to its global phase term. The phases are removed from the output arms and added to the phase shifts of the sMZI, as seen in Fig. 3.8 (ii). We can continue this propagation until the phase reaches the edge of the mesh. Hence, we can use the phase shifters at the edges to generate the required intermediate phases.

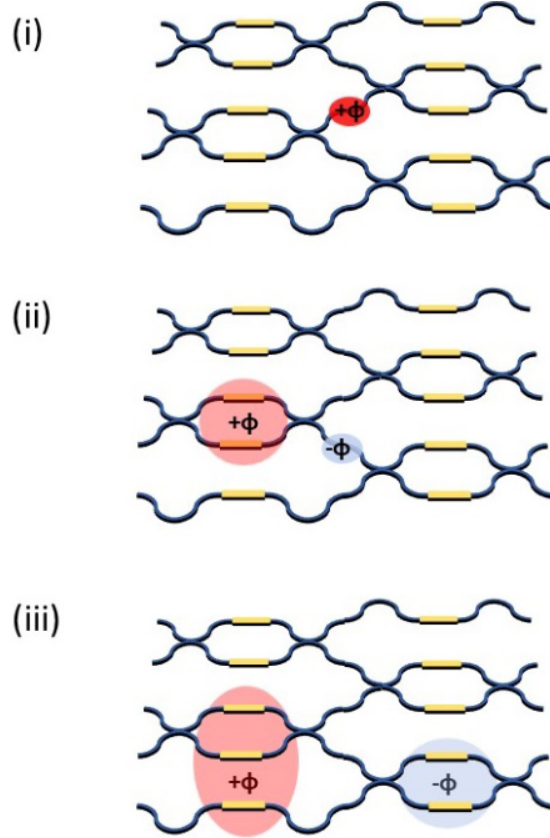


Figure 3.8: Procedure for moving intermediate phase shift to the edges of the mesh [29]

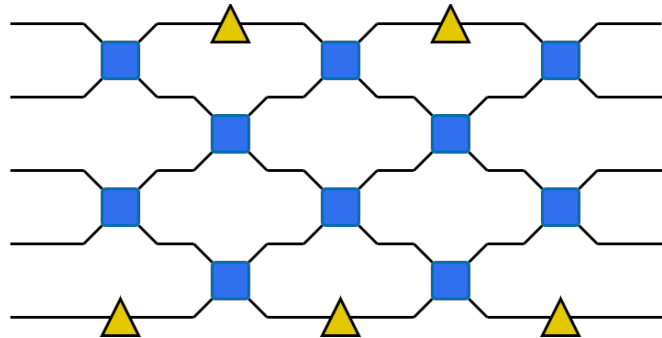


Figure 3.9: Compact version of Clements mesh with relocated phase shifters (external shifters omitted)

Fig. 3.9 shows the compact version of the 5×5 Clements mesh in Fig. 3.7 after the relocation of phase shifters. This structure closely resembles the unitary blocks of our SVD chip. Instead of empty waveguides, the edges of the mesh have attenuators (Fig.

3.4). The transmission function, which can be rewritten as $e^{i\omega} \cos \delta$, allows us to set the required phase shifts by varying ω .

Thus, we have shown the equivalence between aMZI and sMZI meshes. Utilising the sMZI-based arrangement facilitates a more efficient and compact design, enabling easier scaling and potentially reducing losses associated with long waveguide paths.

METHODS

4.1 Mesh Calibration

The discussion on calibration techniques for MZI meshes in [Section 2.3](#) indicated that most of the existing calibration methods are not directly applicable to the SVD mesh. Hence, a customized algorithm or framework tailored to address the challenges posed by the SVD chip is required.

In MZI mesh networks, mitigating static phase errors emerges as a practical approach to enhancing performance. Among various calibration strategies, global optimization methods stand out due to their model-independent nature, allowing them to treat the photonic circuit as a black box. Hence, global optimisation methods will be at the core of the phase estimation methods discussed in this study.

4.1.1 Optimisation Approach

The objective of our calibration process is to accurately estimate the phase errors in an $N \times N$ mesh. We denote the phase errors as $(\theta_e, \phi_e, \gamma_e)$. The mesh's **passive transformation matrix**, which represents the configuration where all phase shifters are **inactive**, is denoted by A_p . This matrix A_p is inherently affected by the phase errors.

Let $\hat{A}_p(\theta, \phi, \gamma)$ represent the ideal transformation matrix of the mesh. The calibration challenge involves minimizing the loss function $\mathcal{L}$, which quantifies the discrepancy between A_p and $\hat{A}_p(\theta, \phi, \gamma)$:

$$\text{Minimize } \mathcal{L}(A_p, \hat{A}_p(\theta, \phi, \gamma))$$

The hypothesis states that the phases that minimize the loss function should approximate the phase errors in the mesh,

$$(\hat{\theta}, \hat{\phi}, \hat{\gamma}) \approx (\theta_e, \phi_e, \gamma_e)$$

For phase error estimation, we only need to perform a "single pass" to determine the passive transmission matrix. Once that is measured, the remaining procedure happens on computer. Fig. 4.1 shows the flow in which operations occur.

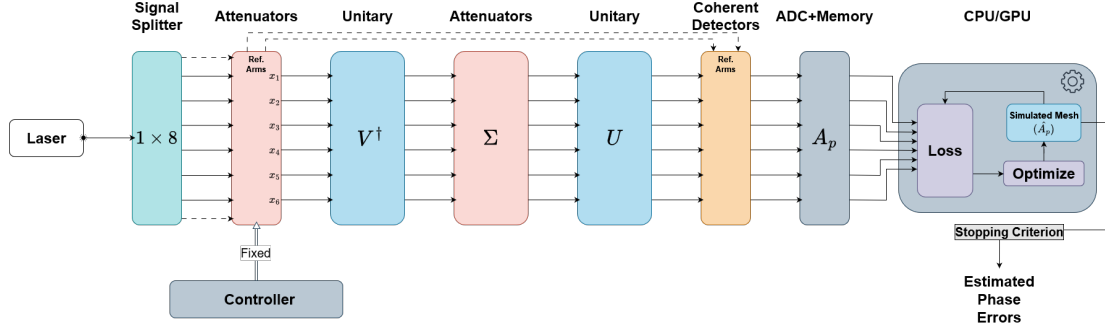


Figure 4.1: Phase Error Estimation in SVD chip using Global Optimisation

For loading a target matrix onto the chip, a similar procedure needs to be followed. Let A denote the target matrix and $\hat{A}(\theta, \phi, \gamma)$ denote the estimated transmission matrix (the transmission matrix of the chip). The loss between A and $\hat{A}(\theta, \phi, \gamma)$ needs to be minimized for successful loading.

$$\text{Minimize } \mathcal{L}(A, \hat{A}(\theta, \phi, \gamma))$$

The merit of this method lies in the fact that it compensates for any fabrication errors in the mesh, since the phases are fine-tuned to bring the transmission matrix as close as possible to the target. Also, notice that mesh loading involves multiple passes of the mesh, when compared phase error estimation. Fig. 4.2 visualizes the procedure for mesh loading. The loss function used in the experiments is the **Frobenius Norm Loss**,

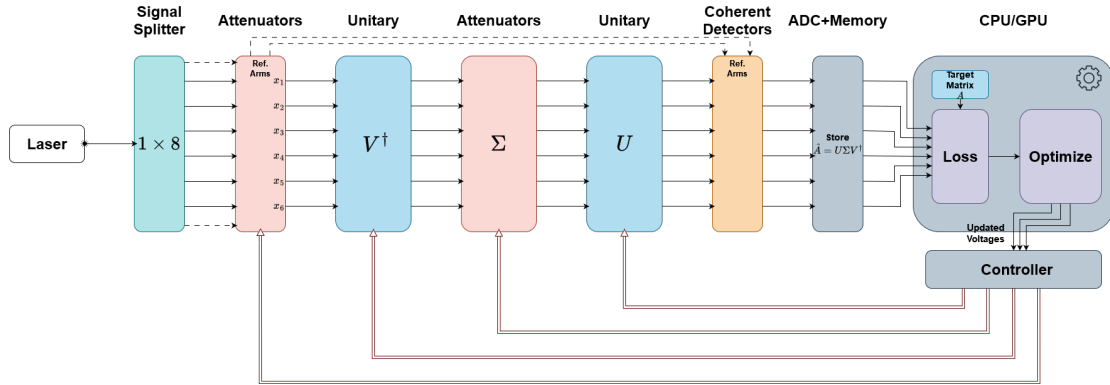


Figure 4.2: Loading a target matrix onto the SVD chip using Global Optimisation

given by,

$$\mathcal{L}_F(A, B) = \frac{1}{N^2} \sum_{i,j} |A_{ij} - B_{ij}|^2 \quad (4.1)$$

Initial experiments were conducted on 6×6 unitary blocks to validate the feasibility of the calibration method. This preliminary phase allowed us to refine our approach before scaling up to the full SVD mesh.

For the phase error estimation simulations, we emulate the real phase errors in the mesh by sampling these errors from a Normal distribution with zero mean and standard deviation σ . In other terms, $\theta_e \sim N(0, \sigma_\theta^2)$. This standard deviation provides the ‘scale’ of the phase errors in the system. Its value is chosen to be within 10% of the max phase setting in the simulation.

4.1.2 Optimisation Methods

The minimization of the loss function was conducted using two distinct optimization methods, **Gradient Descent** and **Particle Swarm Optimisation (PSO)** [30].

Gradient descent is a popular method that is preferred due to its simplicity, proven convergence and low computational cost. Out of all gradient based optimizers, the **Adam** optimizer was utilised in the experiments since it empirically converged the fastest. It integrates the benefits of two other extensions of stochastic gradient descent: Adaptive Gradient Algorithm (AdaGrad) and Root Mean Square Propagation (RMSProp), making it highly effective for handling complex, high-dimensional spaces such as those in MZI meshes.

PSO is a robust, evolutionary algorithm inspired by the social behaviors observed in birds and fish. It is particularly effective for optimizing photonic meshes because it adeptly navigates through various potential configurations. Each solution, or "particle," in the swarm adjusts its position based on personal experience and the influence of its neighbors. This adjustment is driven by the best solution a particle has encountered (cognitive component, c_1) and the best solution observed by any particle in its vicinity (social component, c_2). The inertia weight (w) moderates the influence of previous velocity on current adjustments, helping to maintain momentum.

The convergence of PSO is highly dependent on the initial distribution of particles and the parameters governing their interaction, making it crucial to fine-tune these settings for specific applications in photonic circuit design. It tends to get stuck at local minimas. There is no proof of convergence, which is something to keep in mind.

4.1.3 Unitary Isolation Algorithm

The below algorithm is proposed to isolate the unitary block U from the influence of $V^\dagger$, so that we can independently calibrate the U block and work backwards to $V^\dagger$, thereby calibrating the entire mesh.

Recall that the Σ block represents a diagonal matrix with terms of the form $e^{i\omega} \cos \delta$, where ω and δ are calculated from θ_1 and θ_2 . Let δ' be the actual phase shift induced (includes the phase errors). The Σ matrix of the chip will be a function of δ' . However, the only parameters under our control are the voltages that produce δ_{applied} . Hence, we will rewrite the Σ matrix as $\hat{\Sigma}(\delta_{\text{applied}})$.

Let $\hat{y}$ be the output of the SVD mesh. Let x be the signal right after the attenuators of

the splitting tree. Then,

$$\hat{y} = U \cdot \hat{\Sigma}(\delta_{\text{applied}}) \cdot V^\dagger \cdot x$$

Algorithm Overview:

1. Program $V^\dagger$ and U (to the best of our ability) such that there is power is directed to all the outputs. Fix their configuration.
2. Send in adequate power to the input port of the chip. Make sure all photodetectors detect non-zero power. Fix the input attenuators configuration.
3. Optimise the phase shifters of the Σ block such that the photodetectors yield zero output.

$$\text{Minimize } \|\hat{y}(\delta_{\text{applied}})\|^2 = x^\dagger V |\hat{\Sigma}(\delta_{\text{applied}})|^2 V^\dagger x$$

This is equivalent to finding $\hat{\delta}_{\text{applied}}$ such that $\hat{\Sigma}(\hat{\delta}_{\text{applied}}) = 0$.

4. Once $\hat{\delta}_{\text{applied}}$ is found, phase error in δ can be estimated by noticing that $\delta' = \frac{\pi}{2}$ when nullification occurs in the attenuator. Hence,

$$\hat{\delta}_e = \left| \frac{\pi}{2} - \hat{\delta}_{\text{applied}} \right|$$

Once the errors are determined, we can control the magnitude of Σ and thereby the inputs to U . Fig. 4.3 shows a visual summary of the proposed algorithm.

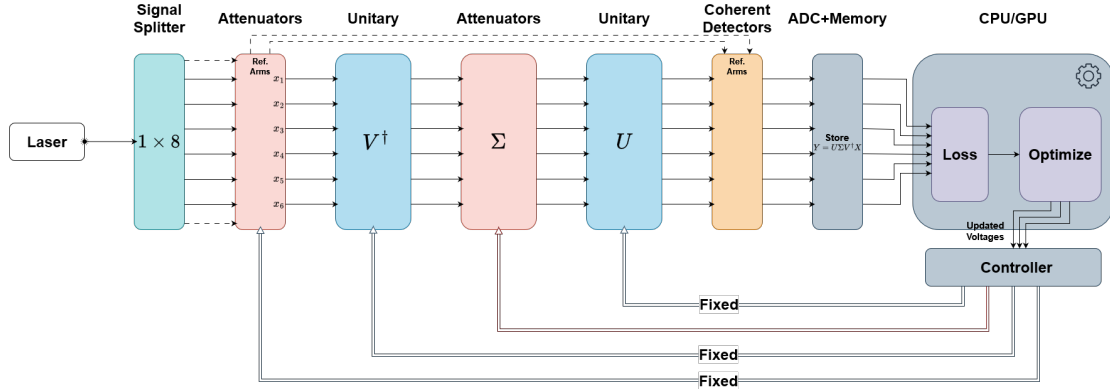


Figure 4.3: The flow of operations in the Unitary Isolation Algorithm

4.2 Training an Optical Neural Network

Once chip calibration is finished, the mesh is ready to be used as a layer in an ONN. It is not immediately apparent whether using ideal photonic meshes as layers in a neural network affects its performance. Hence, the evaluation of an ONN on handwritten digit classification task is done. The results are contrasted to results of conventional neural networks, assessing the differences in training and testing phases.

The data is obtained from the MNIST handwritten digits database. The dataset is a large collection of handwritten digits that is widely used for training and testing machine learning models.

- The dataset consists of grayscale images of handwritten digits (0 through 9).
- Each image is 28×28 pixels, with each pixel value being an integer between 0 and 255, with 0 representing black and 255 representing white.
- The dataset contains 60,000 images for the training phase and 10,000 images for the testing/evaluation phase. All images come with an associated label that indicates which number is drawn in the image.

A feedforward fully connected neural network containing photonic mesh layers will be used for classification.

4.2.1 Preprocessing

Since we're not using a Convolutional Neural Network to do image classification, the 28×28 images need to be flattened before we can send them as an input to the network. However, having $28^2 = 784$ inputs leads to a lengthy mesh layer. As simulating this is infeasible, there is a need for size reduction.

Rather than resorting to pooling operations, a Fourier transform-based processing technique was utilised [31]. Upon performing a 2D DFT of the images in our dataset, we observe most of active pixels were around the low frequencies. This makes sense, since there isn't much periodic variation along any direction of our image. Hence, we can crop out the inactive regions and reduce our image size. This is like performing a low pass filter on an image and then downsampling.

The additional advantage of this method is that it exploits the complex nature of the meshes. The mesh matrix is a complex matrix, and multiplying it with a complex input can yield fruitful results.

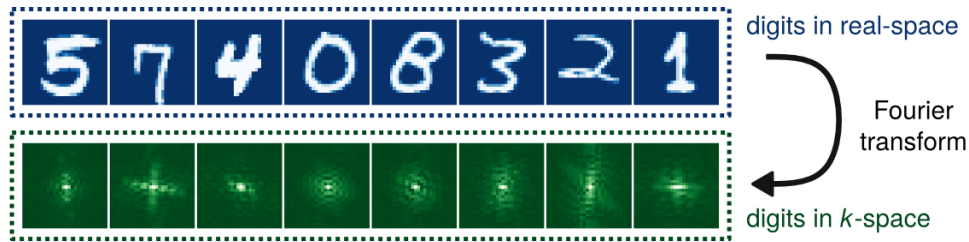


Figure 4.4: *Plots of the 2D DFT of the images*

4.2.2 Network Architecture

The ONN resembles a fully connected feedforward network with each hidden layer simulated using a photonic mesh. The choice of photonic mesh can either be a unitary or an SVD mesh. The performance of both types of meshes were recorded and compared. It's expected that the SVD mesh would outperform the unitary mesh as the weight matrix is unrestricted.

An L layer ONN needs to have activation functions between each layer. There is active research being done to create purely optical non-linear activation functions [8, 31]. The activation function also needs to handle complex values. modReLU [32] is a popular activation function for complex data. The function is defined as,

$$\sigma_{\text{modReLU}}(z) = \begin{cases} (|z| + b) \frac{z}{|z|} & \text{if } |z| + b > 0 \\ 0 & \text{if } |z| + b \leq 0 \end{cases}$$

where $b \in \mathbb{R}$ is a bias parameter that can be fixed or learnt. There are electro-optic devices that can realise a transformation similar to modReLU. Hence, modReLU is activation function of choice in the multi-layer ONN experiments.

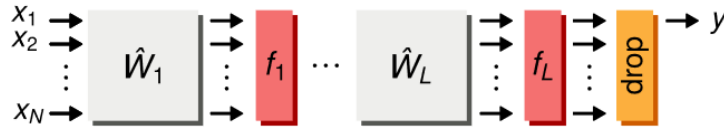


Figure 4.5: Network architecture in terms of mesh layers

Since convergence is sensitive to initial conditions, the phase initialisation is done using the Haar random scheme [26] for θ , and uniform random for the rest. For tasks that require M outputs, we can drop the remaining output modes and train the network. There will be no backpropagation from these modes, hence the phases of the meshes will be learnt accordingly. Also, the output activation function performs L2 normalisation on the outputs, since the network is used for classification.

4.2.3 Multi-core ONNs

In MZI-based ONNs, the size of the ONN scales with the size of the input. Specifically, for data characterized by M features, an ONN requires a mesh comprising M input waveguides or channels to facilitate matrix multiplication. Despite efforts in feature selection, the dimensionality of the features can remain high, commonly reaching into the thousands. To date, the largest experimental realization of an MZI mesh is 64×64 [10]. This limitation indicates that such setups are only feasible for applications involving equal or smaller scales.

To facilitate higher-dimensional matrix computations on processors of smaller dimensions, one can leverage the concept of block matrices. This approach involves partitioning large matrices into smaller submatrices, where the operations on these blocks adhere to the conventional rules of matrix algebra, given that the block matrices are conformable for multiplication. Specifically, for a matrix M that is $2N \times 2N$, it can be decomposed into four $N \times N$ submatrices. The representation is as follows,

$$M = \left[\begin{array}{c|c} A & B \\ \hline C & D \end{array} \right]$$

where A, B, C , and D are each $N \times N$ matrices. The product of two such block matrices, say M and another matrix N of similar structure, can be computed under the matrix multiplication rules,

$$M \cdot N = \left[\begin{array}{c|c} A & B \\ \hline C & D \end{array} \right] \cdot \left[\begin{array}{c|c} E & F \\ \hline G & H \end{array} \right] = \left[\begin{array}{c|c} AE + BG & AF + BH \\ \hline CE + DG & CF + DH \end{array} \right]$$

This method not only aligns with the traditional matrix multiplication rules but also significantly enhances the computational scalability of optical processors. In the context of the SVD mesh, a single $2N \times 2N$ chip can be replaced using four $N \times N$ chips. The MVM is done as follows,

$$M \cdot x = \left[\begin{array}{c|c} A & B \\ \hline C & D \end{array} \right] \cdot \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} Ax_1 + Bx_2 \\ Cx_1 + Dx_2 \end{bmatrix} \quad (4.2)$$

where x is the input to the chip which is split into x_1 and x_2 of equal sizes. Fig. 4.6 shows the mapping between a neural network and the multi-core ONN implementation, as demonstrated in [33]. The calculation output of each PCNNC (Photonic Complex-valued Neural Network Chip) is combined on computer using Eq. 4.2.

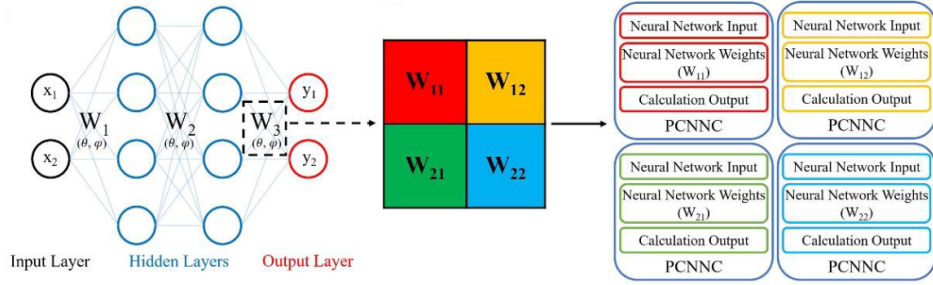


Figure 4.6: Multi-core implementation of ONNs [33]

4.2.4 Robustness Analysis

After identifying the optimal phase configuration for the ideal ONN, it is crucial to conduct a thorough analysis of the system's tolerance to phase noise. The phase noise is modelled as a Gaussian White Noise to accurately represent its random and wideband characteristics.

In order to simulate the effects of noise, the phase errors are sampled from a Normal distribution with zero mean and standard deviation of ϵ . These phase errors are overlayed onto the optimal phases of the mesh and the weight matrices are recomputed. The final step in this process involves evaluating the performance of the perturbed ONN by performing inference on the test dataset. The accuracy of the classification is measured and plotted for various degrees of ϵ . Also, the robustness of single-core chips is contrasted to that of multi-core chips, to check if there's merit in using smaller chips.

RESULTS

All the methods listed in the previous chapter were tested using simulations. Neurophox is an open source Python-based machine learning and photonics simulation framework. Modifications were made to this framework to suit our experiments and a custom version of this package can be found [here](#). The code for these simulations can be found in the `calibration.ipynb` in [this repository](#).

Neurophox assumes the aMZI Clements mesh while creating the meshes. However, it was shown in [Section 3.3](#) that a conversion scheme exists when going one type of mesh to another. Hence, we can simulate on these meshes without any issues.

5.1 Phase Error Estimation

All the experiments assume zero beamsplitter errors. Phase error σ was set to 10% of maximum phase setting for all the experiments in this section.

5.1.1 Gradient Descent

Simulation 1: Unitary

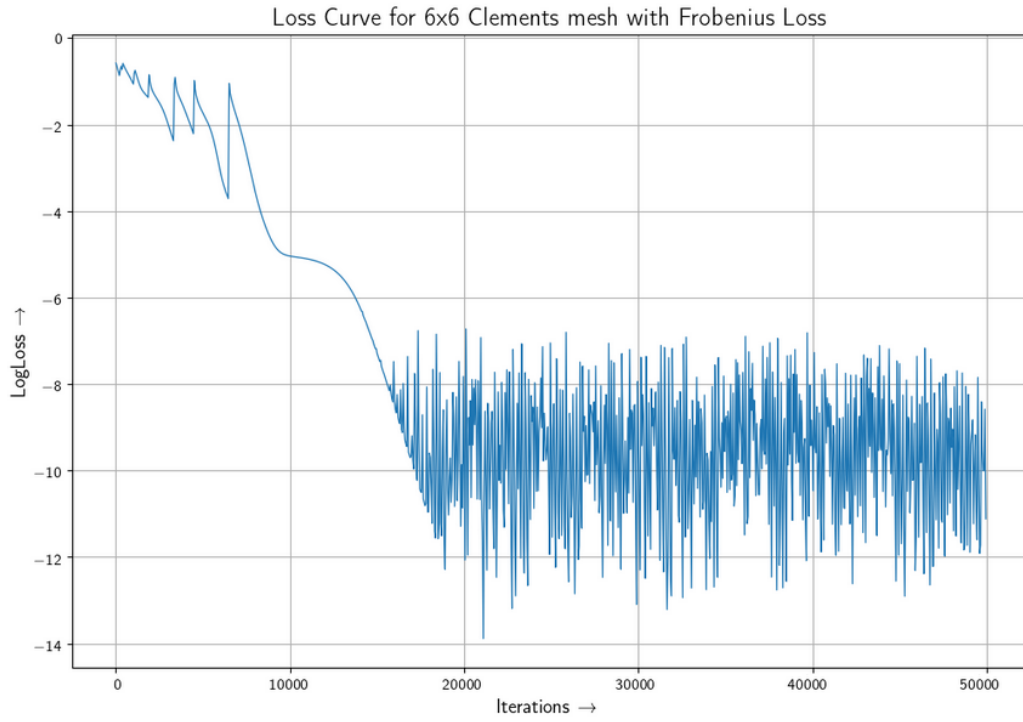
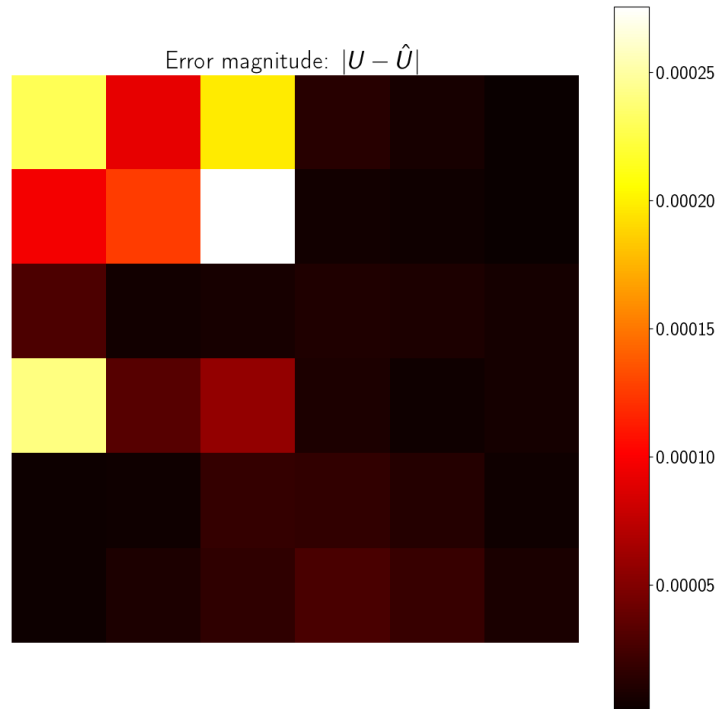
This simulation was optimized using Adam with the following hyperparameters:

- Learning Rate = 0.002
- Iterations = 5×10^4

Table 5.1: Deviation (in radians) between Actual Phase Error and Estimated Phase Error for Unitary with Frobenius Loss = 1.299×10^{-14}

Phase	Max Deviation (Δ_{max})	Min Deviation (Δ_{min})	Mean Deviation (Δ_{mean})
θ	$\sim 10^{-5}$	$\sim 10^{-7}$	$\sim 10^{-5}$
ϕ	$\sim 10^{-4}$	0	$\sim 10^{-5}$
γ	$\sim 10^{-5}$	0	$\sim 10^{-6}$

Table 5.1 shows the deviation Δ in phase errors, which is defined as the absolute difference between actual and estimate. Using this method, the phase errors are

(a) *Loss curve*(b) *Absolute error in matrix estimate***Figure 5.1:** *Results of phase error estimation for unitary mesh*

estimated until 4 decimal places. From Fig. 5.1a, we can see that it takes about 20000 iterations to reach minimum loss. If we were loading the matrix onto the mesh, it would take 6 forward passes of the mesh to store the current matrix configuration into memory. Hence, it would take 1.2×10^5 shots to load the matrix.

Simulation 2: SVD

Convergence in SVD meshes is not as straightforward as it is in unitary meshes. The minimum loss is sensitive to the learning rate of the Adam optimizer. Too high of a learning rate can cause the loss to drop rapidly at the beginning but fail to converge in the later steps. Table 5.2 shows the minimum loss obtained for various learning rate settings.

Table 5.2: Minimum loss vs. Learning Rate of Adam for SVD mesh

Learning Rate	Minimum Loss
0.0007	$\sim 10^{-4}$
0.001	$\sim 10^{-5}$
0.002	$\sim 10^{-8}$
0.005	$\sim 10^{-6}$
0.0075	$\sim 10^{-5}$
0.01	$\sim 10^{-5}$

Thus, the hyperparameters for this simulation are set to the following:

- Learning Rate = 0.002
- Iterations = 10^5

Table 5.3: Deviation (in radians) between Actual Phase Error and Estimated Phase Error for SVD mesh with Frobenius Loss = 1.12×10^{-7}

U block				
Phase	Max Deviation (Δ_{max})	Min Deviation (Δ_{min})	Mean Deviation (Δ_{mean})	$\Delta \leq 3^\circ$
θ	0.309	$\sim 10^{-5}$	8.91×10^{-2}	7/15
ϕ	6.09	5.75×10^{-2}	2.87	0/15
γ	5.50	3.42	4.71	0/6
$V^\dagger$ block				
Phase	Max Deviation (Δ_{max})	Min Deviation (Δ_{min})	Mean Deviation (Δ_{mean})	$\Delta \leq 3^\circ$
θ	3.03×10^{-2}	$\sim 10^{-5}$	$\sim 10^{-3}$	15/15
ϕ	5.49	$\sim 10^{-5}$	0.517	12/15
γ	4.50	0.205	1.86	0/6
Σ block				
Phase	Max Deviation (Δ_{max})	Min Deviation (Δ_{min})	Mean Deviation (Δ_{mean})	$\Delta \leq 3^\circ$
δ	$\sim 10^{-4}$	$\sim 10^{-7}$	$\sim 10^{-5}$	6/6

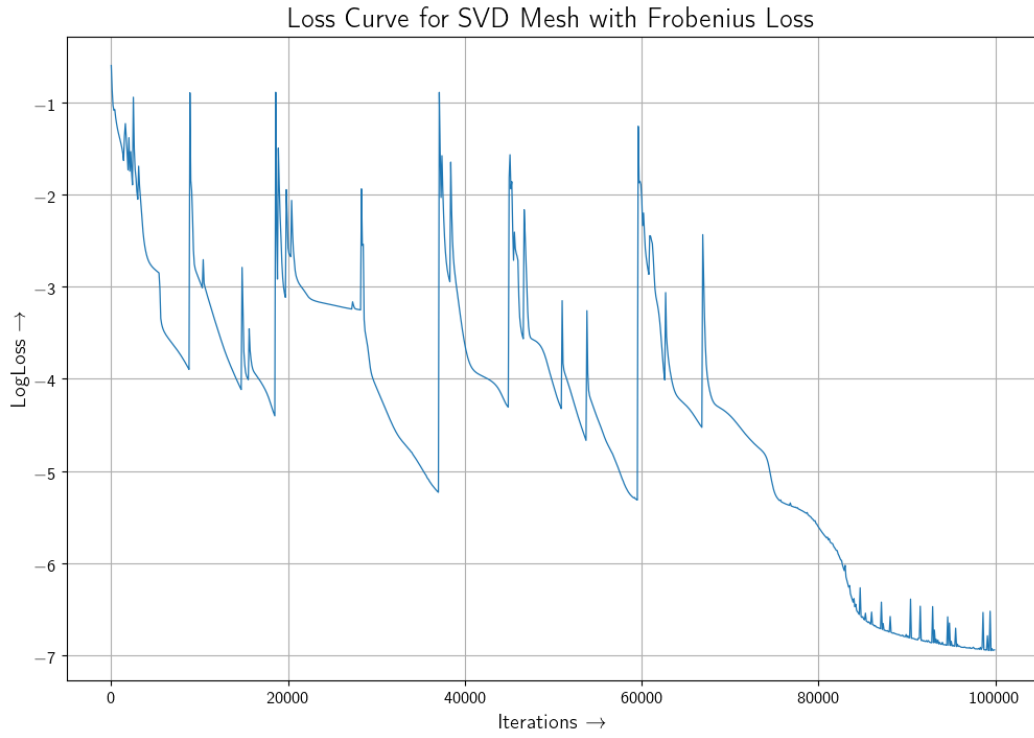
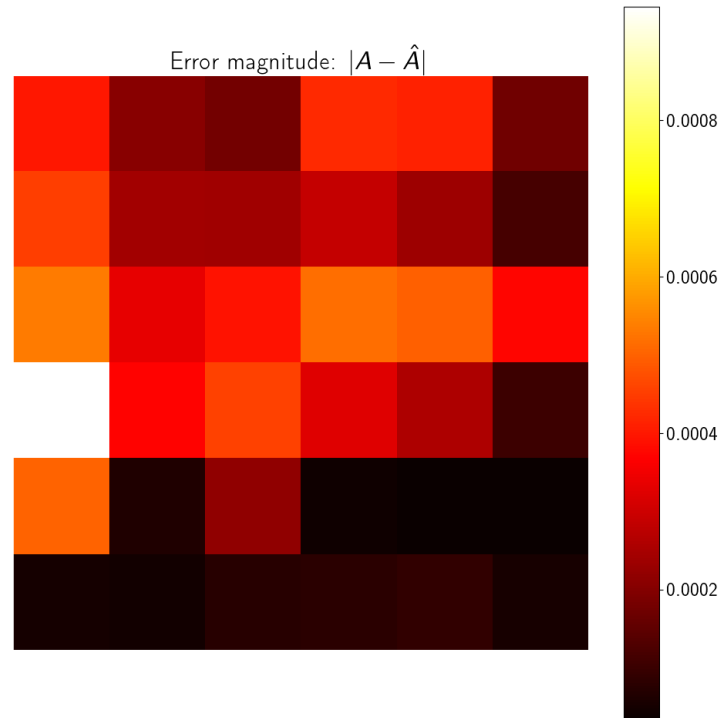
(a) *Loss curve*(b) *Absolute error in matrix estimate***Figure 5.2:** *Results of phase error estimation for SVD mesh*

Table 5.3 showcases the deviation in phase error estimates. The column labelled $\Delta \leq 3^\circ$ counts the number of phase error estimates with deviation less than 3° . This gives us a criterion for the number of phase errors that have been correctly estimated.

While the loss is relatively low (Fig. 5.2b), the phase errors are not fully determined. θ seems to be estimated much more than ϕ , which makes sense considering the fact that the unitaries in the SVD of a matrix are unique upto a phase factor. γ is not estimated correctly at all, whereas the Σ phase errors are determined until 4 decimal places.

Simulation 3: SVD with bounded phases

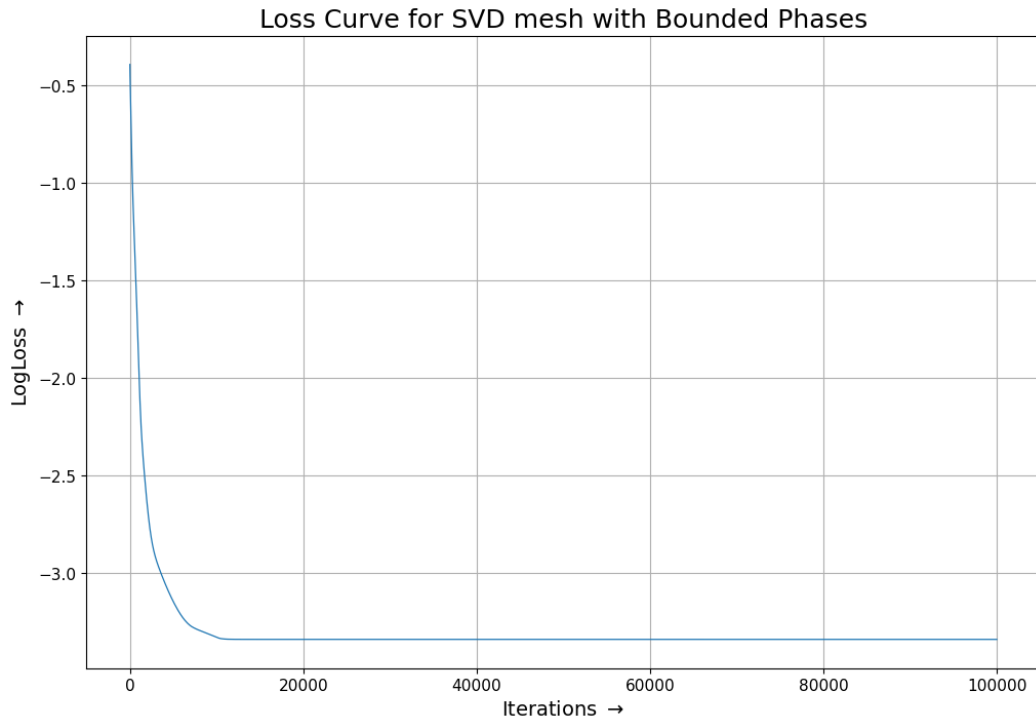
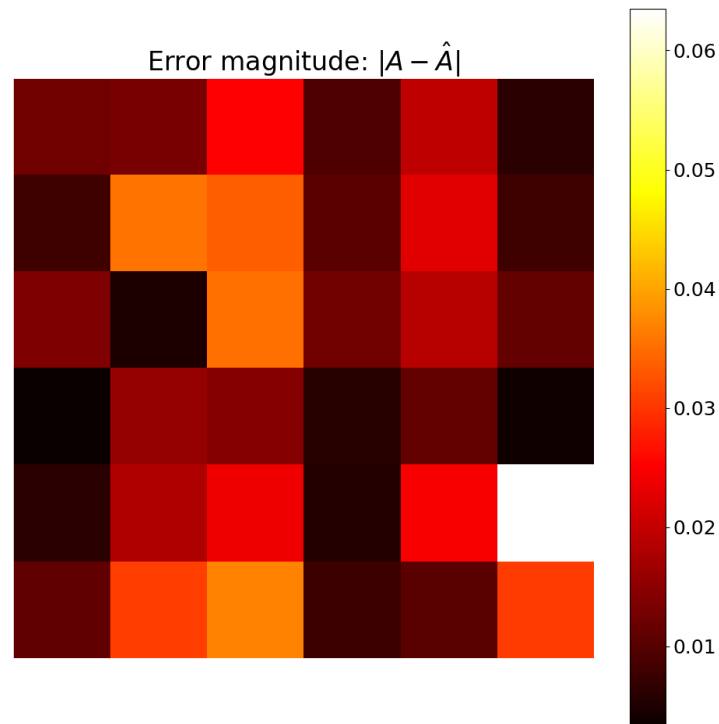
In this simulation, the phases were bounded to small values (30% of their maximum), since the phase errors we're trying to estimate aren't very large. The hypothesis is that bounding the phases could reduce the solution space down to a unique solution, which makes it possible to determine errors in ϕ .

This simulation was optimized using Adam with the following hyperparameters:

- Learning Rate = 0.001
- Iterations = 10^5

Table 5.4: Deviation (in radians) between Actual Phase Error and Estimated Phase Error for SVD mesh with Frobenius Loss = 4.57×10^{-4}

U block				
Phase	Max Deviation (Δ_{max})	Min Deviation (Δ_{min})	Mean Deviation (Δ_{mean})	$\Delta \leq 3^\circ$
θ	0.522	$\sim 10^{-3}$	0.0765	9/15
ϕ	0.546	0.0236	0.188	2/15
γ	0.364	0.137	0.221	0/6
$V^\dagger$ block				
Phase	Max Deviation (Δ_{max})	Min Deviation (Δ_{min})	Mean Deviation (Δ_{mean})	$\Delta \leq 3^\circ$
θ	0.134	$\sim 10^{-4}$	0.0349	12/15
ϕ	0.117	0.00149	0.0394	10/15
γ	0.284	0.00425	0.0932	3/6
Σ block				
Phase	Max Deviation (Δ_{max})	Min Deviation (Δ_{min})	Mean Deviation (Δ_{mean})	$\Delta \leq 3^\circ$
δ	$\sim 10^{-3}$	$\sim 10^{-4}$	$\sim 10^{-3}$	6/6

(a) *Loss curve*(b) *Absolute error in matrix estimate***Figure 5.3:** *Results of phase error estimation for SVD mesh with bounded phases*

On comparing Table 5.4 with Table 5.3, we can see that while the estimates in block U have slightly improved, the estimates in V block have gotten poorer. Some γ s in block V have been estimated. Although it may seem like further training could improve these results, the loss curve in Fig. 5.3a shows saturation. Bounding the phases seems to make it easier to fall into a local minima.

5.1.2 PSO

In PSO, the following parameters are in our control:

- N_p : Number of particles
- c_1 : Cognition learning rate
- c_2 : Social learning rate
- w : Inertial weight
- $(x_{\min}, x_{\max})$: Bounds on particle position
- M : Number of iterations

The simulations were performed for a 6×6 unitary first, since the search space is smaller. A range of parameter values were tried in order to estimate the phase errors.

First, the relation between loss and number of particles N_p was investigated. In order to search the 36 dimensional parameter space adequately, higher number of particles might be a necessity. The upper bound on the particle position is set to 30% of the maximum phase setting.

Table 5.5: *Minimum Loss versus N_p*

N_p	c_1	c_2	w	N_{iter}	Min. Loss
40	0.5	0.3	0.9	2500	0.0161
100	0.5	0.3	0.9	1000	0.0207
400	0.5	0.3	0.9	500	0.0139
1000	0.5	0.3	0.9	100	0.0125
10000	0.5	0.3	0.9	100	0.0169

A random search was performed on the hyperparameters (c_1, c_2, w) . The search was run through 150 configurations, with $c_1 \in [0.2, 1.5]$, $c_2 \in [0.2, 1.5]$ and $w \in [0.4, 2]$. Every optimisation was done for $N_p = 40$ and 500 iterations. However, the best result of this search was $(c_1, c_2, w) = (0.931, 0.373, 1.66)$, with a loss of 0.0253, which is greater than the loss in Table 5.5. The high computation times coupled with marginal loss reduction makes this method inferior to gradient descent.

5.1.3 Unitary Isolation Algorithm

Simulation 1: Σ Calibration

This simulation involves the minimisation of the mesh output by varying the Σ block attenuator phases. `scipy.minimize` is a standard tool for optimisation, and was used for this small scale optimisation problem. The loss function, which is just MSE, was minimized using **COBYLA**, a gradient-free numerical method for constrained optimisation. The optimisation is done over the applied phase δ . The phase errors are generated as before, with $\sigma = 0.1$.

The optimization ended with a loss of 9.77×10^{-6} in 677 iterations. Hence, minimization is possible without much effort.

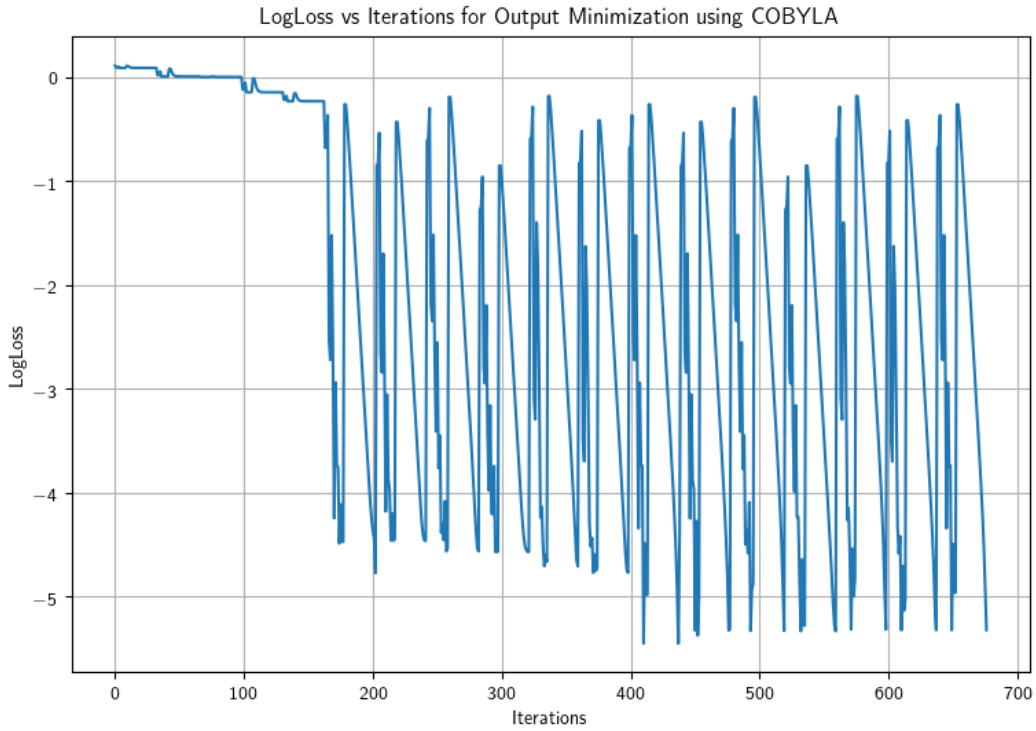


Figure 5.4: Loss Curve for Mesh Output Minimization using COBYLA

To verify the nullification condition, we compute the estimated phase error $\hat{\delta}_e = \left| \frac{\pi}{2} - \hat{\delta}_{\text{applied}} \right|$ and compare it to the actual phase error δ_e .

Table 5.6: Applied Phase ($\hat{\delta}_{\text{applied}}$), Estimated Phase Error ($\hat{\delta}_e$) and Actual Phase Error (δ_e) of the Σ block (in radians)

Phases	Mode 1	Mode 2	Mode 3	Mode 4	Mode 5	Mode 6
$\hat{\delta}_{\text{applied}}$	1.59036441	1.58367956	1.57474809	1.5865862	1.57574571	1.59308812
$\hat{\delta}_e$	0.01956809	0.01288323	0.00395176	0.01578987	0.00494939	0.02229179
δ_e	0.0195692	0.01287729	0.00394799	0.01578223	0.00494711	0.02229611

Table 5.6 confirms the nullification condition and shows that the errors have been determined successfully.

Since the Σ block is calibrated, the global optimisation method no longer needs to tune these values. This reduction in degrees of freedom could improve the error estimates in the U and $V^\dagger$.

Simulation 2: U & $V^\dagger$

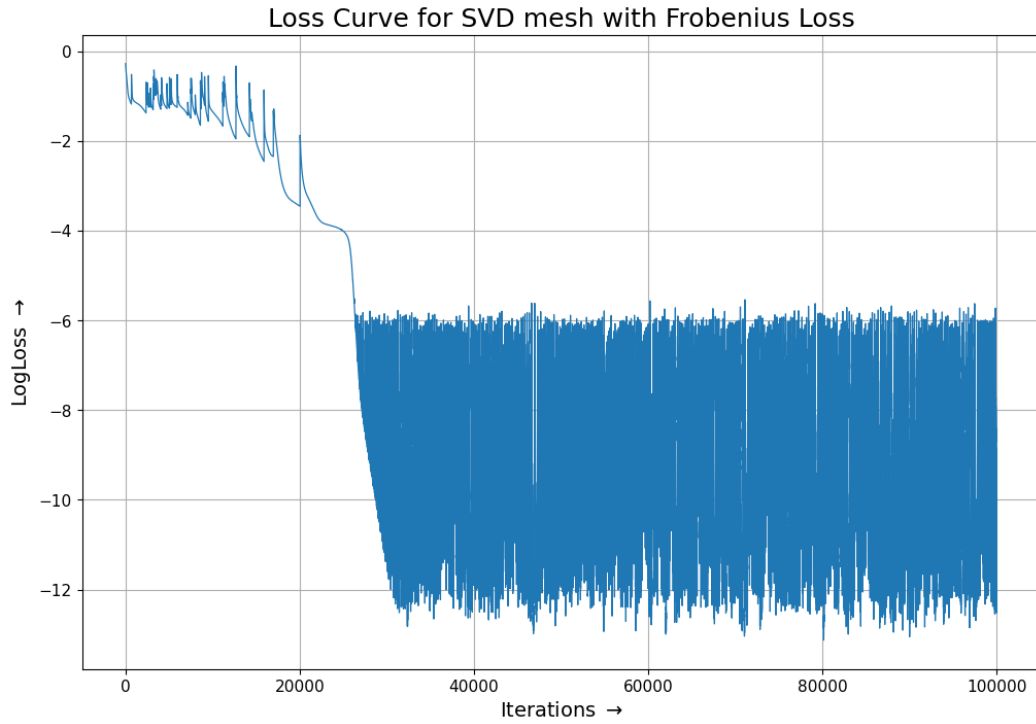
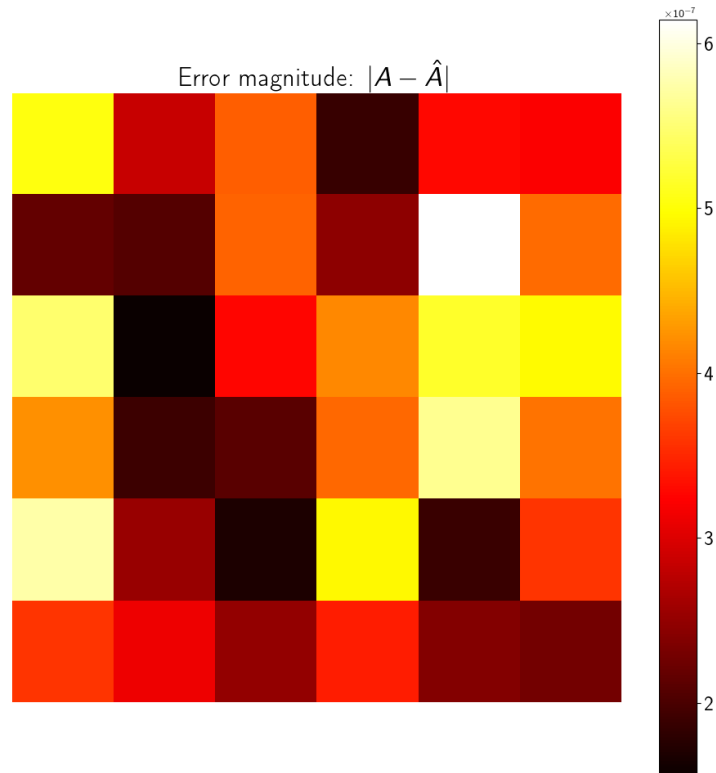
This simulation freezes the phases of Σ and only trains on the phases of the U and $V^\dagger$ blocks. This simulation was optimized using Adam with the following hyperparameters:

- Learning Rate = 0.002
- Iterations = 10^5

Table 5.7: Deviation (in radians) between Actual Phase Error and Estimated Phase Error of U and $V^\dagger$ blocks with Frobenius Loss = 7.357×10^{-14}

U block				
Phase	Max Deviation (Δ_{max})	Min Deviation (Δ_{min})	Mean Deviation (Δ_{mean})	$\Delta \leq 3^\circ$
θ	$\sim 10^{-6}$	$\sim 10^{-7}$	$\sim 10^{-6}$	15/15
ϕ	4.89	$\sim 10^{-7}$	1.05	5/15
γ	4.84	1.439	2.01	0/15
$V^\dagger$ block				
Phase	Max Deviation (Δ_{max})	Min Deviation (Δ_{min})	Mean Deviation (Δ_{mean})	$\Delta \leq 3^\circ$
θ	$\sim 10^{-6}$	$\sim 10^{-8}$	$\sim 10^{-6}$	15/15
ϕ	$\sim 10^{-5}$	$\sim 10^{-9}$	$\sim 10^{-6}$	15/15
γ	4.84	0.426	1.72	0/15

The loss curve and matrix error plot in Fig. 5.5 show that losses and errors are very low. Table 5.7 shows that $V^\dagger$ is almost fully calibrated. Since γ values only affect the phase of the output (Eq. 3.2), we have effectively isolated the U block, since the power of the signals coming into U are completely determined by the transformation $\Sigma V^\dagger$, which is known. This opens up the possibility of employing calibration techniques uniquely tailored for Unitary meshes, which were previously inapplicable to the entire chip.

(a) *Loss curve*(b) *Absolute error in matrix estimate (colourmap scale has an order of magnitude of -7)***Figure 5.5:** Results of phase error estimation for SVD mesh with calibrated Σ

5.2 Optical Neural Network

Before analyzing the ONN simulation outcomes, it is essential to establish a benchmark for comparison. A basic fully connected neural network (FCNN) with 784 inputs, one hidden layer of 50 neurons using ReLU activation, was trained on the MNIST dataset using TensorFlow. This model served as the baseline for comparing the ONN results. The model was trained for 100 epochs using Adam optimizer with default learning rate.

The highest testing accuracy achieved through this network is 96.3%, as shown in Fig. 5.6. This is a point of reference that will be used to judge the effectiveness of ONNs.

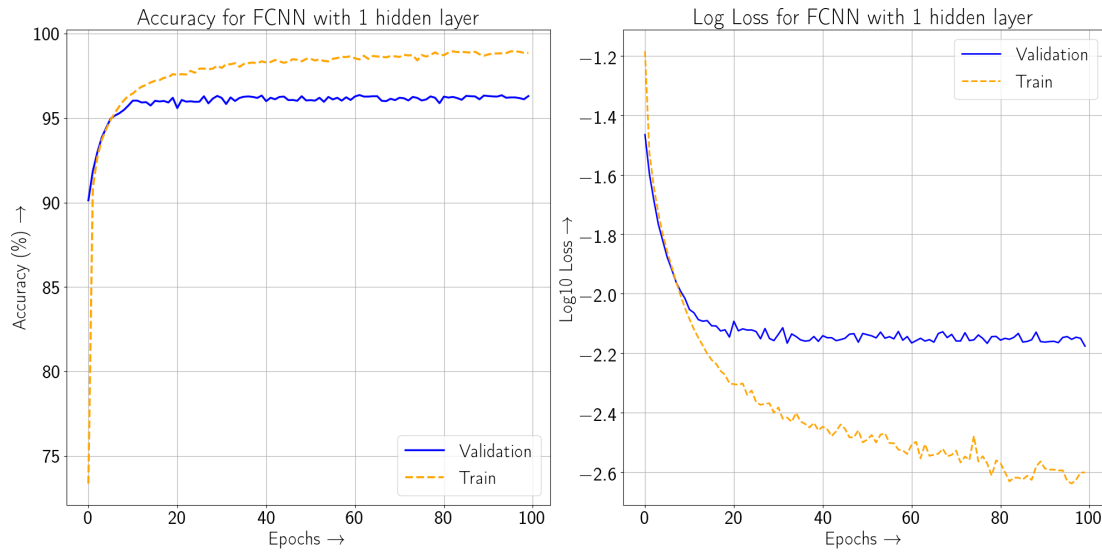


Figure 5.6: Results of the conventional FCNN training

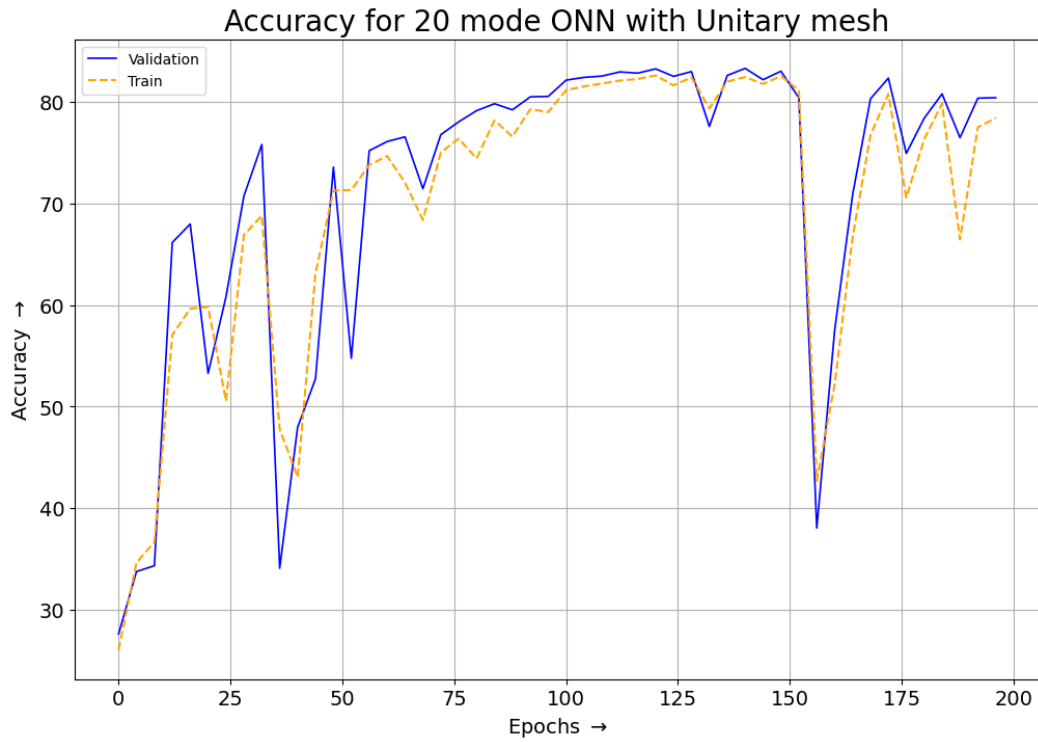
5.2.1 Classification Results

Simulation 1: ONN with Unitary Mesh

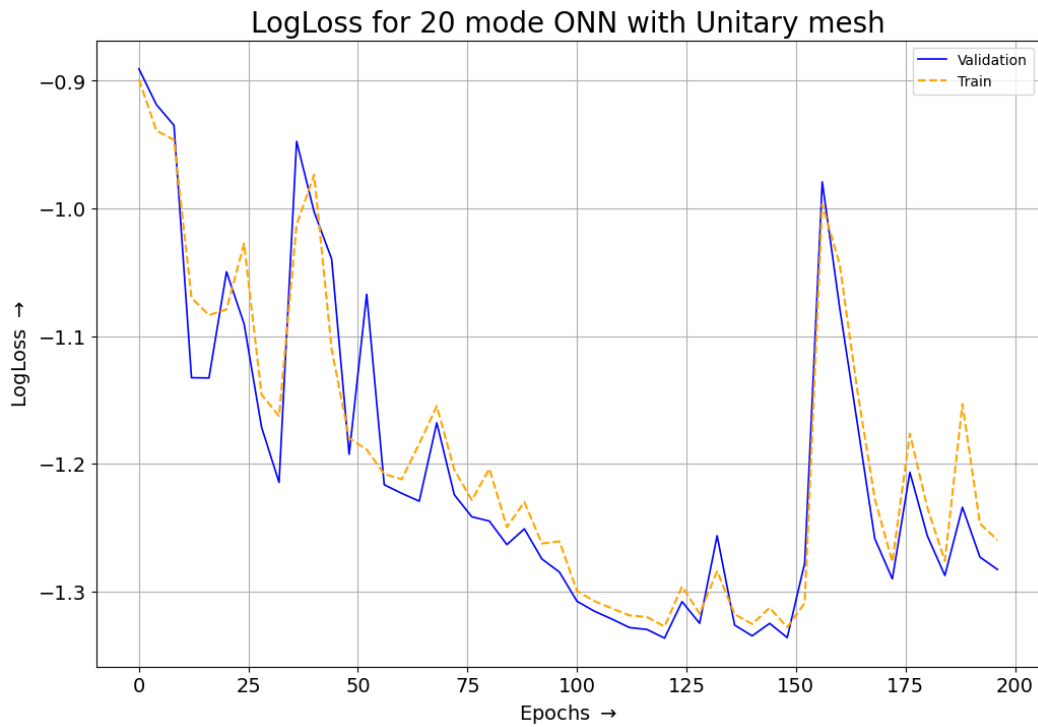
This simulation uses a single 20 mode Unitary mesh to simulate a 20 input fully connected neural network with 1 hidden layer. There is no activation function present after the mesh. The hyperparameters of this network are:

- Loss Function = Mean Squared Error
- Learning Rate = 0.002
- Epochs = 200
- Batch Size = 512

Fig. 5.7 displays the results of training. The best accuracy attained is 83.40% at epoch 149.



(a) Accuracy vs. Epochs



(b) Loss Curve

Figure 5.7: Results of single layer unitary mesh ONN training

Simulation 2: ONN with SVD Mesh

This simulation uses a single 20 mode SVD mesh to simulate a 20 input fully connected neural network with 1 hidden layer. There is no activation function present after the mesh. The hyperparameters of this network are:

- Loss Function = Mean Squared Error
- Learning Rate = 0.002
- Epochs = 200
- Batch Size = 512

Fig. 5.8 showcases the training results. The best accuracy attained is 86.14% at epoch 78. This is an improvement over the Unitary mesh (Fig. 5.7a). Hence, the ability of the SVD mesh to load an arbitrary weight matrix is favourable in ONNs.

Simulation 3: Accuracy vs. N

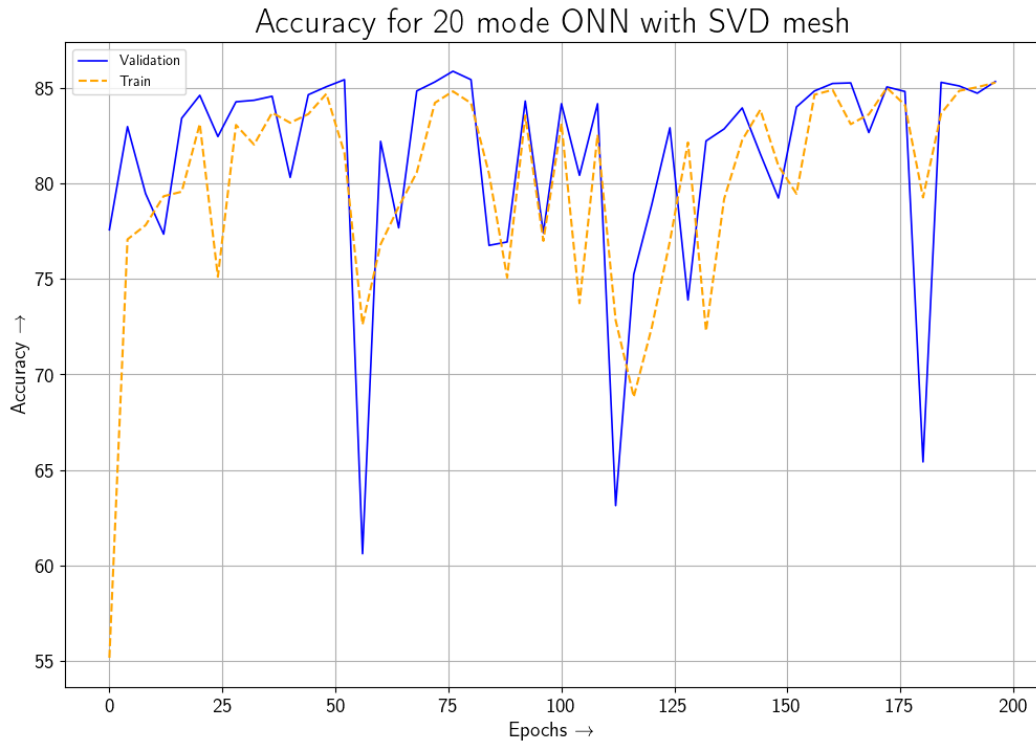
To observe the relation between accuracy and mesh size, the dataset was preprocessed at different sizes and fed to Unitary mesh ONNs of varying sizes. There is no activation present after the mesh in these simulations. All hyperparameters were fixed to the following values:

- Number of mesh layers (L) = 1
- Loss Function = Mean Squared Error
- Learning Rate = 0.002
- Epochs = 200
- Batch Size = 512

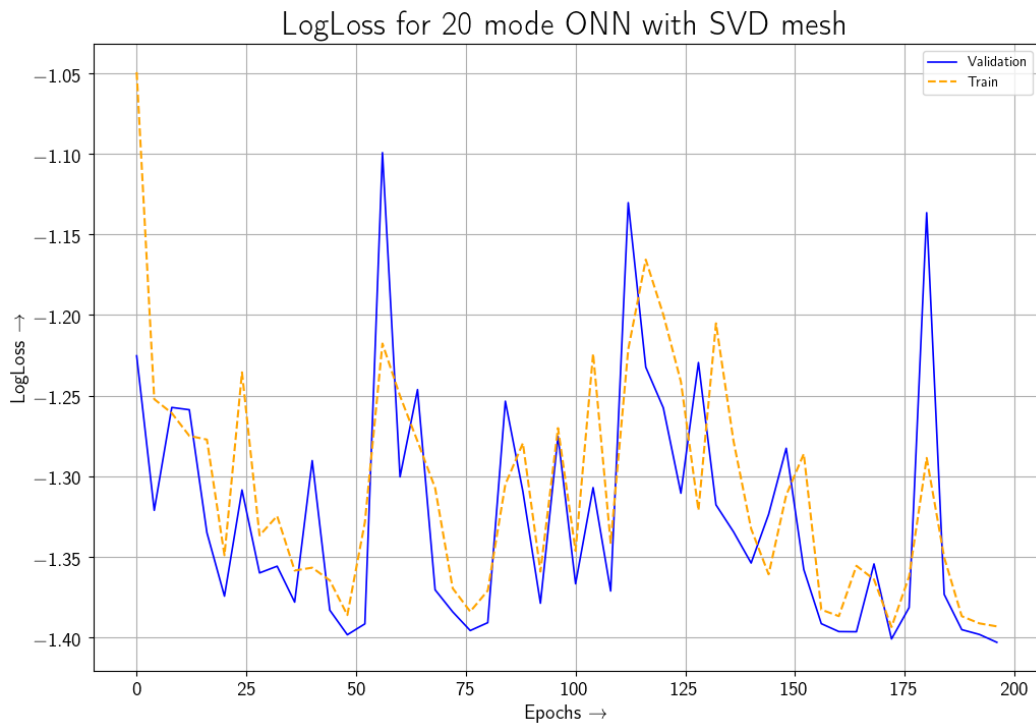
Table 5.8: Accuracy vs. N (Epochs = 200 and $L=1$)

Input Dim (N)	Train Accuracy (%)	Test Accuracy (%)
20	82.65	83.40
36	87.80	89.57
49	91.23	91.80
64	91.83	92.30
100	84.61	88.62

There was an upwards trend between testing accuracy and N , but the $N = 100$ case didn't perform as expected. The large number of parameters might need more epochs to converge. However, the losses were oscillating rather than trending downward. The observed rise in accuracy with larger image sizes may stem from reduced information loss in the images during preprocessing. Further simulations on other datasets are needed to understand this.



(a) Accuracy vs. Epochs



(b) Loss Curve

Figure 5.8: Results of single layer SVD mesh ONN training

Simulation 4: Increasing L and Activation Function

To observe the performance of a deeper network with a non-linear activation function, conditions identical to those in Simulation 1 and 2 were recreated. The present simulations aimed to model a fully connected neural network with 20 inputs, two hidden layers, and an activation function.

Two optical neural networks (ONNs) were developed, utilizing Unitary and Singular Value Decomposition (SVD) layers, respectively. Both networks share identical hyperparameters:

- Number of mesh layers (L) = 2
- Loss Function = Mean Squared Error
- Learning Rate = 0.002
- Epochs = 200
- Batch Size = 512

Table 5.9: Accuracy Comparisons for Different Network Configurations

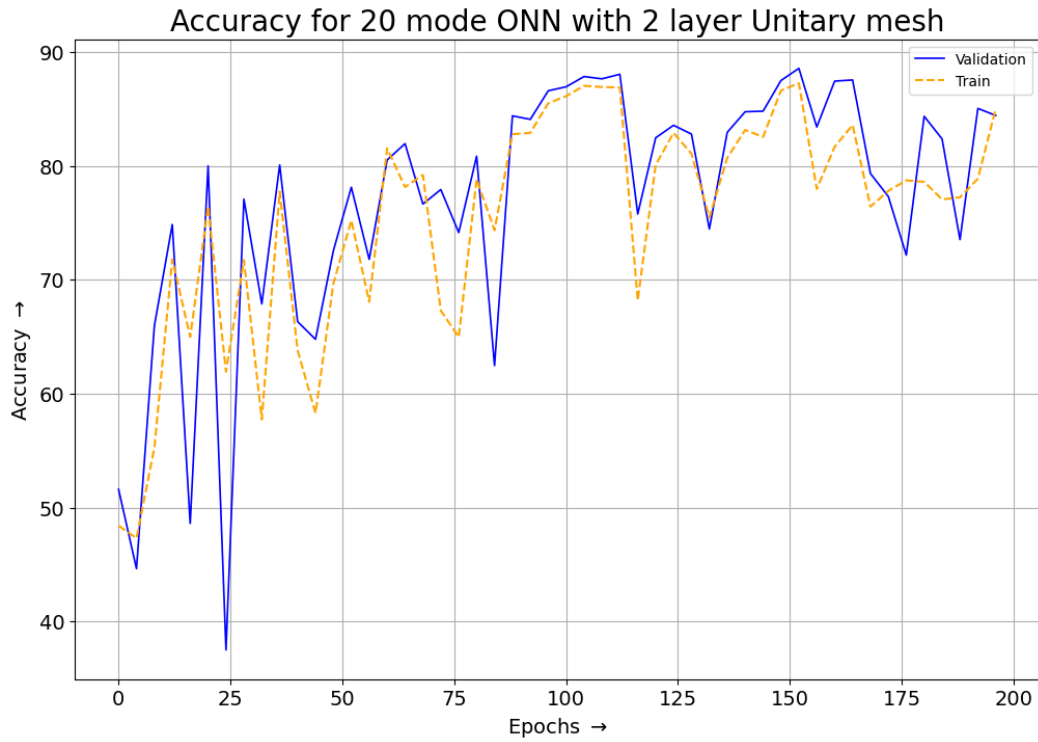
No. of Mesh Layers (L)	Unitary		SVD	
	Train Acc (%)	Test Acc (%)	Train Acc (%)	Test Acc (%)
1	82.65	83.40	85.13	85.53
2	87.26	88.57	90.58	91.42

Table 5.9 summarises the results obtained for different configurations and showcases the effect of increasing L . There is a roughly 5-6 % increasing in all accuracy metrics when the second layer is added. This indicates that increasing the network depth could improve the accuracy of ONNs. Also, the performance of SVD mesh is still better than unitary mesh even after adding a layer.

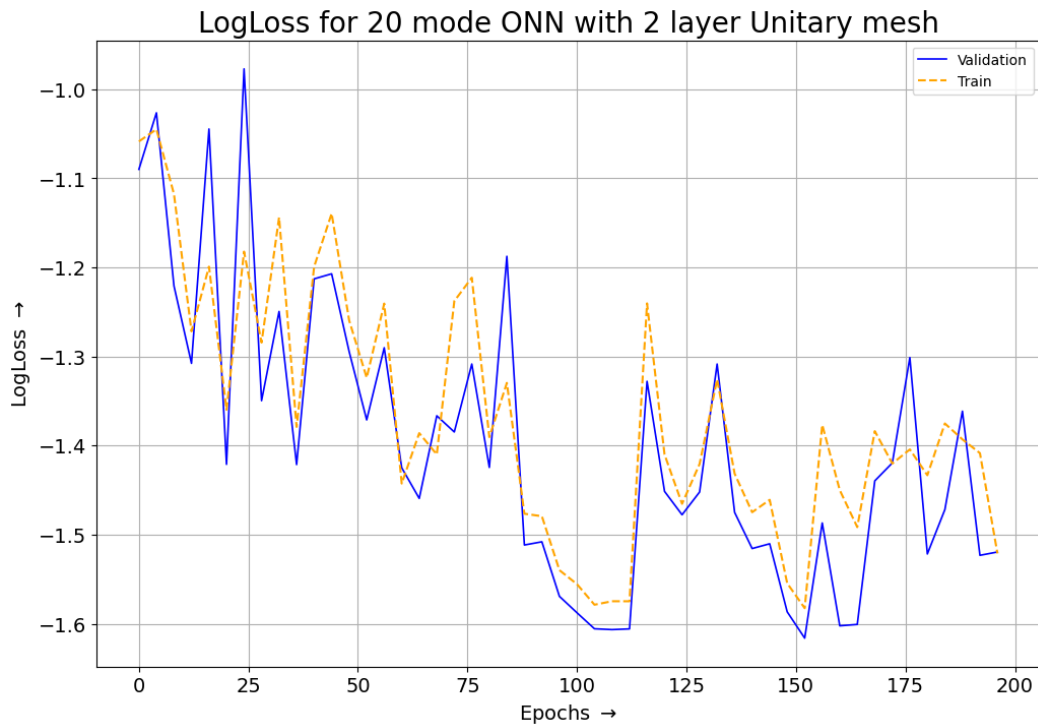
Simulation 5: Multi-core ONN

A single-core 16×16 SVD mesh chip can be replaced a multi-core chip with four 8×8 SVD meshes. Two single layer ONNs with these respective chips are trained and evaluated. Apart from the mesh layers, the hyperparameters are the same as previous simulations.

Fig. 5.11 shows the result of training these ONNs. The accuracies are comparable for both chips, with the multi-core ONN giving 89.3% and single-core giving 89.1%. This shows that multi-core implementations are capable of giving on par or better performance than single-core.

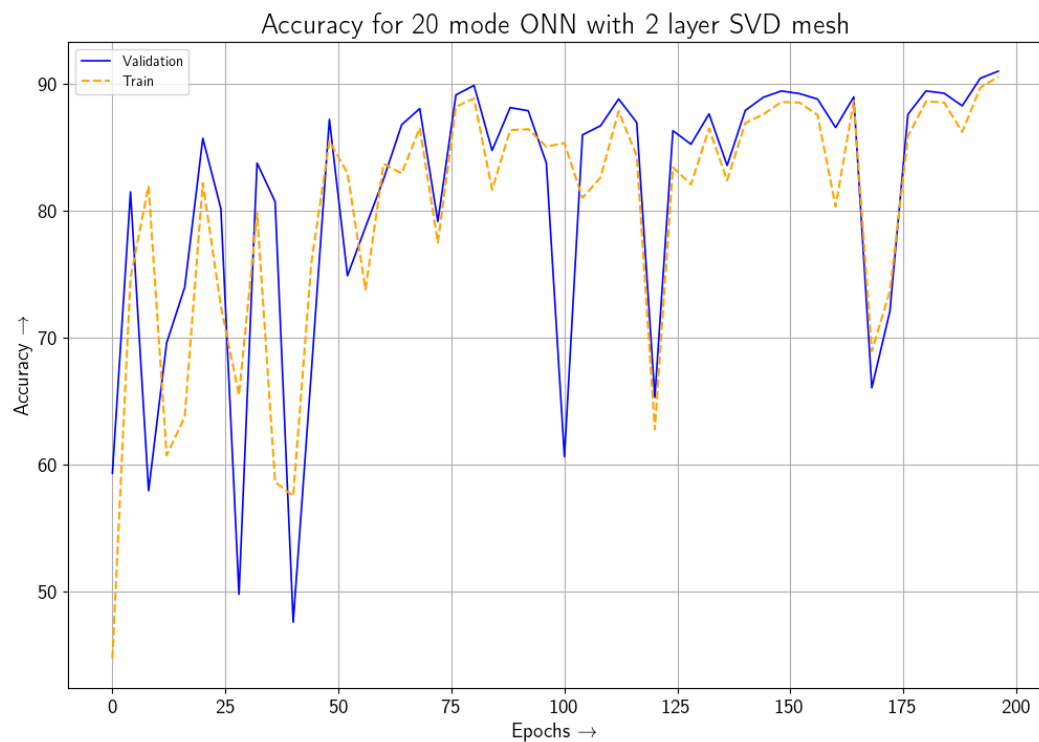


(a) Accuracy vs. Epochs

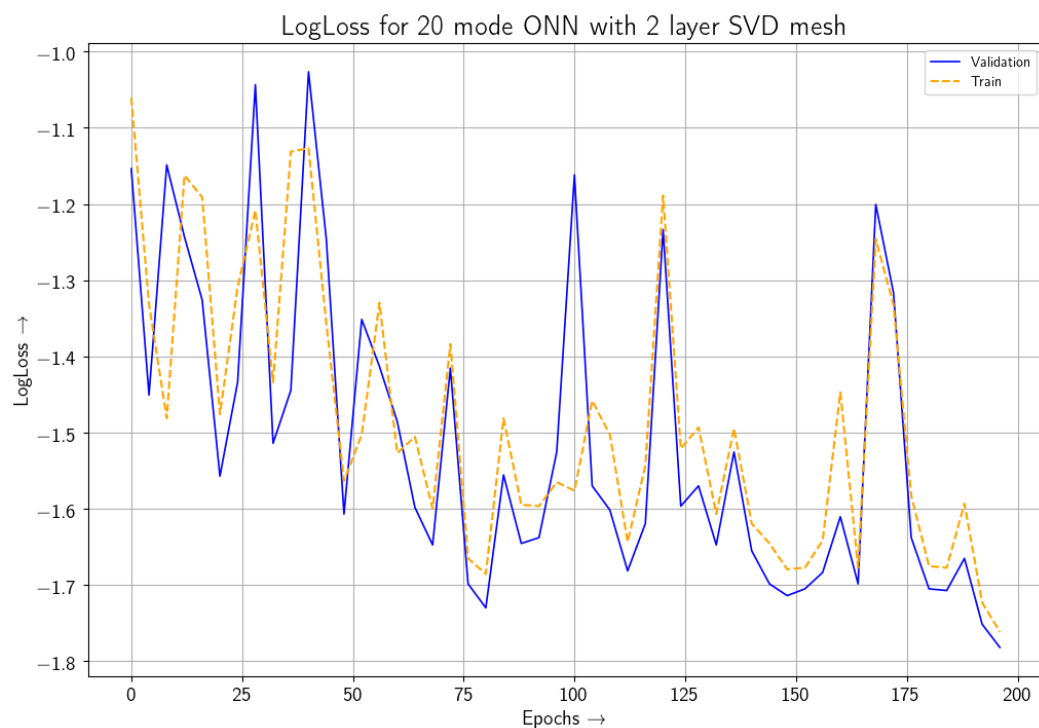


(b) Loss Curve

Figure 5.9: Results of 2 layer unitary mesh ONN training

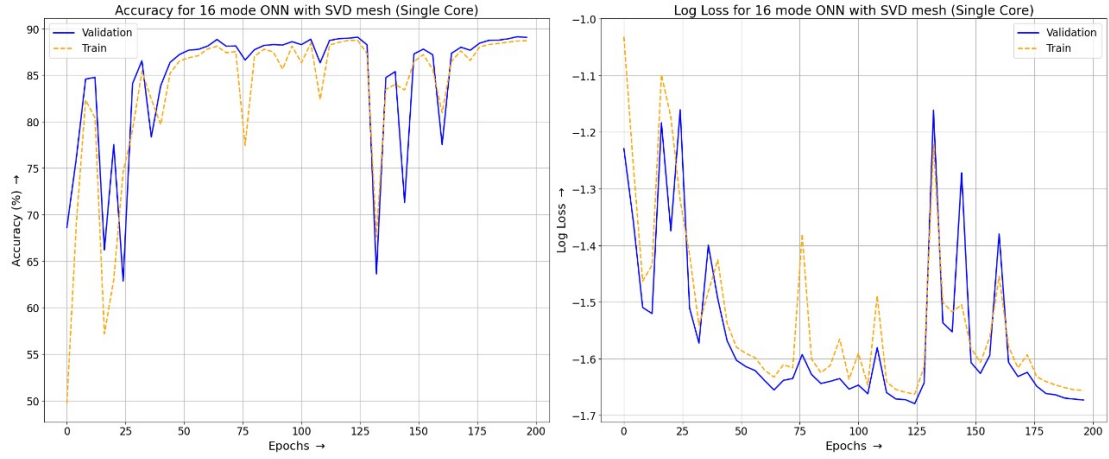


(a) Accuracy vs. Epochs

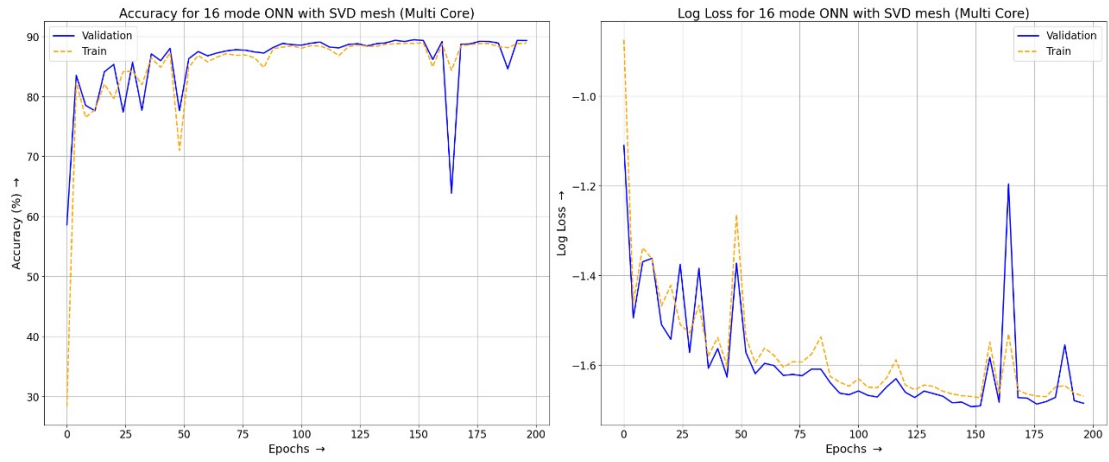


(b) Loss Curve

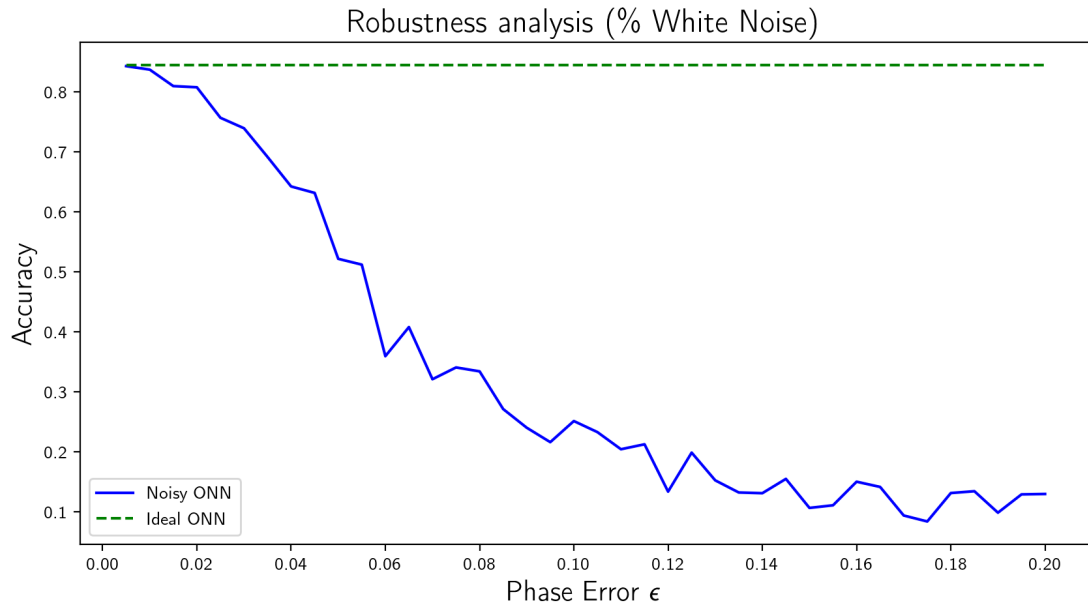
Figure 5.10: Results of 2 layer SVD mesh ONN training



(a) Results of the single-core 16 mode ONN training



(b) Results of the multi-core 16 mode ONN training

Figure 5.11: Single-core vs. multi-core chip performance**Figure 5.12:** Accuracy vs. Phase Error ϵ (rad) in a 20 mode unitary ONN

5.2.2 Robustness Analysis

The robustness to phase errors was checked by deliberately introducing phase errors to the phases of a trained mesh layer. The model from Simulation 1, depicting the performance of an optimal mesh, served as the reference standard. In the absence of any errors, this benchmark model achieved an accuracy of 83.4%, corresponding to a configuration with $N = 20$, $L = 1$, and 200 training epochs. The introduced phase errors, denoted by ϵ , varied from 0.01 to 0.2 radians.

Fig. 5.12 shows how the accuracy deteriorates sharply as the phase error increases. An approximately 10% drop in accuracy occurs when phase errors are of the scale $\epsilon \approx 0.03$ radians. As we can see, the ONN is very sensitive to phase errors. These results underscore the importance of robust calibration techniques for real MZI meshes.

Fig. 5.13 shows the robustness analysis of single-core and multi-core implementations. The multi-core chip gives better accuracies than single-core for all phase error scales. This could be because of the lower optical depth of smaller meshes, reducing the propagation of errors at the output. These results demonstrate the advantage of multi-core implementation of ONNs.

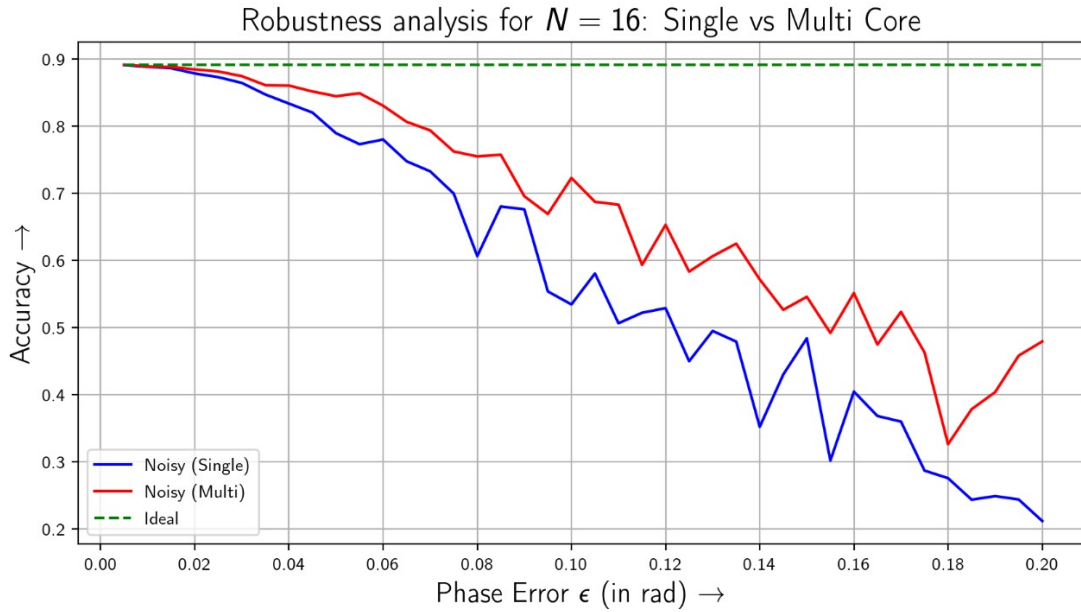


Figure 5.13: Accuracy vs. Phase Error ϵ (rad) for 16 mode single and multi-core ONNs

CONCLUSION AND FUTURE WORK

In this study, several strategies for calibrating ONNs were discussed and evaluated, yielding insights into their effectiveness and limitations. Global optimization techniques for calibrating unitary meshes proved to be highly effective, successfully identifying all phase errors, which underscores their potential utility for precise ONN calibration tasks. However, when these methods were extended to SVD meshes, the results were mixed. The estimation of θ errors was successful, yet challenges arose with ϕ and γ errors due to the inherent non-uniqueness of the unitaries obtained from SVD upto a phase factor.

Particle Swarm Optimization (PSO) did not perform adequately in calibrating unitary meshes, as indicated by significantly higher loss values compared to those achieved via gradient descent. This finding suggests that PSO may not be suitable for this large of a mesh, even with extensive tuning of hyperparameters. However, there are several PSO variants that come with their own advantages and use cases. These variants could be better suited for mesh calibration.

In contrast, the Unitary Isolation approach introduced in this report offers a promising direction for SVD mesh calibration. Upon effectively calibrating the Σ block, gradient descent was performed on the unitary blocks. This led to semi-calibration of the $V^\dagger$ block, which allows for further calibration of the downstream U block through standard methods discussed in the literature review.

The ONN simulations exhibited commendable performance in digit classification tasks, achieving accuracies exceeding 83%, with peak performance nearing 93%. These results approach the benchmark accuracy of 96.3% and suggest that optimizing the parameters N and L could further enhance ONN performance, potentially matching or surpassing conventional benchmarks. Multi-core ONNs are capable of giving similar performance to single-core ONNs in the ideal simulations. This proves that large dimensional MVMs can be performed using multiple smaller optical processors.

Robustness analysis further highlighted the critical need for effective mesh calibration, as minor phase errors as small as 0.05 radians caused significant accuracy degradation,

reinforcing the importance of developing more fault tolerant calibration methods. Additionally, multi-core ONNs are more resilient to phase noises, potentially due to smaller optical depth of the individual MZI meshes.

There a lot more avenues left to be explored. A few of the ideas and tasks are listed below.

- Change the underlying mesh structure for Neurophox to create a 1:1 simulation of the 6×6 SVD chip.
- Incorporate thermal crosstalk and phase drift into the mesh models.
- Simulate the approach by Hamerly et al. in [24] and contrast its performance with our current approaches.
- Construct an experiment that loads the desired matrix onto the SVD chip, as shown in Fig. 4.2.
- Test the calibration techniques experimentally and gauge the extent of hardware errors in the chip.

BIBLIOGRAPHY

- [1] Andrew Lohn and Micah Musser. “AI and Compute: How Much Longer Can Computing Power Drive Artificial Intelligence Progress?” In: (Jan. 2022). doi: <https://doi.org/10.51593/2021CA009>.
- [2] Jia Liu et al. “Research progress in optical neural networks: theory, applications and developments”. en. In: *Photonix* 2.1 (Dec. 2021). Number: 1 Publisher: SpringerOpen, pp. 1–39. issn: 2662-1991. doi: [10.1186/s43074-021-00026-0](https://doi.org/10.1186/s43074-021-00026-0).
- [3] Xiubao Sui et al. “A Review of Optical Neural Networks”. In: *IEEE Access* 8 (2020). Conference Name: IEEE Access, pp. 70773–70783. issn: 2169-3536. doi: [10.1109/ACCESS.2020.2987333](https://doi.org/10.1109/ACCESS.2020.2987333).
- [4] H. Zhang et al. “An optical neural chip for implementing complex-valued neural network”. en. In: *Nature Communications* 12.1 (Jan. 2021), p. 457. issn: 2041-1723. doi: [10.1038/s41467-020-20719-7](https://doi.org/10.1038/s41467-020-20719-7).
- [5] James Spall, Xianxin Guo, and A. I. Lvovsky. *Hybrid training of optical neural networks*. arXiv:2203.11207 [physics]. May 2022. doi: [10.48550/arXiv.2203.11207](https://doi.org/10.48550/arXiv.2203.11207).
- [6] Cansu Demirkiran et al. “An Electro-Photonic System for Accelerating Deep Neural Networks”. en. In: *ACM Journal on Emerging Technologies in Computing Systems* 19.4 (Oct. 2023). arXiv:2109.01126 [cs], pp. 1–31. issn: 1550-4832, 1550-4840. doi: [10.1145/3606949](https://doi.org/10.1145/3606949).
- [7] Yichen Shen et al. “Deep learning with coherent nanophotonic circuits”. en. In: *Nature Photonics* 11.7 (July 2017), pp. 441–446. issn: 1749-4885, 1749-4893. doi: [10.1038/nphoton.2017.93](https://doi.org/10.1038/nphoton.2017.93).
- [8] Ying Zuo et al. “All-optical neural network with nonlinear activation functions”. EN. In: *Optica* 6.9 (Sept. 2019). Publisher: Optica Publishing Group, pp. 1132–1137. issn: 2334-2536. doi: [10.1364/OPTICA.6.001132](https://doi.org/10.1364/OPTICA.6.001132).
- [9] Saumil Bandyopadhyay et al. *Single chip photonic deep neural network with accelerated training*. arXiv:2208.01623 [physics]. Aug. 2022. doi: [10.48550/arXiv.2208.01623](https://doi.org/10.48550/arXiv.2208.01623).
- [10] Hailong Zhou et al. “Photonic matrix multiplication lights up photonic accelerator and beyond”. en. In: *Light: Science & Applications* 11.1 (Feb. 2022), p. 30. issn: 2047-7538. doi: [10.1038/s41377-022-00717-8](https://doi.org/10.1038/s41377-022-00717-8).
- [11] Nicola Peserico, Bhavin J. Shastri, and Volker J. Sorger. “Integrated Photonic Tensor Processing Unit for a Matrix Multiply: a Review”. en. In: *Journal of Lightwave Technology* 41.12 (June 2023). arXiv:2211.01476 [physics], pp. 3704–3716. issn: 0733-8724, 1558-2213. doi: [10.1109/JLT.2023.3269957](https://doi.org/10.1109/JLT.2023.3269957).
- [12] Ye Tian et al. “Scalable and compact photonic neural chip with low learning-capability-loss”. en. In: *Nanophotonics* 11.2 (Jan. 2022), pp. 329–344. issn: 2192-8614. doi: [10.1515/nanoph-2021-0521](https://doi.org/10.1515/nanoph-2021-0521).

- [13] Michael Reck et al. "Experimental realization of any discrete unitary operator". en. In: *Physical Review Letters* 73.1 (July 1994), pp. 58–61. ISSN: 0031-9007. DOI: 10.1103/PhysRevLett.73.58.
- [14] William R. Clements et al. "Optimal design for universal multiport interferometers". en. In: *Optica* 3.12 (Dec. 2016), p. 1460. ISSN: 2334-2536. DOI: 10.1364/OPTICA.3.001460.
- [15] Kaveh (Hassan) Rahbardar Mojaver et al. "Addressing the programming challenges of practical interferometric mesh based optical processors". en. In: *Optics Express* 31.15 (July 2023), p. 23851. ISSN: 1094-4087. DOI: 10.1364/OE.489493.
- [16] Farhad Shokraneh, Simon Geoffroy-gagnon, and Odile Liboiron-Ladouceur. "The diamond mesh, a phase-error- and loss-tolerant field-programmable MZI-based optical processor for optical neural networks". en. In: *Optics Express* 28.16 (Aug. 2020), p. 23495. ISSN: 1094-4087. DOI: 10.1364/OE.395441.
- [17] Farhad Shokraneh, Simon Geoffroy-Gagnon, and Odile Liboiron-Ladouceur. "Towards Phase-Error- and Loss-Tolerant Programmable MZI-Based Optical Processors for Optical Neural Networks". en. In: *2020 IEEE Photonics Conference (IPC)*. Vancouver, BC, Canada: IEEE, Sept. 2020, pp. 1–2. ISBN: 978-1-72815-891-4. DOI: 10.1109/IPC47351.2020.9252466.
- [18] Saumil Bandyopadhyay, Ryan Hamerly, and Dirk Englund. "Hardware error correction for programmable photonics". en. In: *Optica* 8.10 (Oct. 2021). arXiv:2103.04993 [physics], p. 1247. ISSN: 2334-2536. DOI: 10.1364/OPTICA.424052.
- [19] David A. B. Miller. "Perfect optics with imperfect components". en. In: *Optica* 2.8 (Aug. 2015), p. 747. ISSN: 2334-2536. DOI: 10.1364/OPTICA.2.000747.
- [20] David A. B. Miller. "Self-configuring universal linear optical component [Invited]". en. In: *Photonics Research* 1.1 (June 2013), p. 1. ISSN: 2327-9125. DOI: 10.1364/PRJ.1.000001.
- [21] Sunil Pai et al. "Experimentally realized in situ backpropagation for deep learning in nanophotonic neural networks". en. In: *Science* 380.6643 (Apr. 2023). arXiv:2205.08501 [physics], pp. 398–404. ISSN: 0036-8075, 1095-9203. DOI: 10.1126/science.ade8450.
- [22] S. Mohammad Reza Safaee, Kaveh Hassan Rahabardar Mojaver, and Odile Liboiron-Ladouceur. "Mitigating Phase Error Accumulation in Programming MZI-Based Optical Processors". en. In: *2023 International Conference on Photonics in Switching and Computing (PSC)*. Mantova, Italy: IEEE, Sept. 2023, pp. 1–3. ISBN: 9798350323702. DOI: 10.1109/PSC57974.2023.10297236.
- [23] Christopher Alexiev et al. "Calibrating rectangular interferometer meshes with external photodetectors". en. In: *OSA Continuum* 4.11 (Nov. 2021), p. 2892. ISSN: 2578-7519. DOI: 10.1364/OSAC.437918.
- [24] Ryan Hamerly, Saumil Bandyopadhyay, and Dirk Englund. "Accurate Self-Configuration of Rectangular Multiport Interferometers". en. In: *Physical Review Applied* 18.2 (Aug. 2022), p. 024019. ISSN: 2331-7019. DOI: 10.1103/PhysRevApplied.18.024019.
- [25] Ryan Hamerly, Saumil Bandyopadhyay, and Dirk Englund. "Asymptotically fault-tolerant programmable photonics". en. In: *Nature Communications* 13.1 (Nov. 2022). Publisher: Nature Publishing Group, p. 6831. ISSN: 2041-1723. DOI: 10.1038/s41467-022-34308-3.
- [26] Sunil Pai et al. "Matrix optimization on universal unitary photonic devices". In: *Physical Review Applied* 11.6 (June 2019). arXiv:1808.00458 [physics], p. 064044. ISSN: 2331-7019. DOI: 10.1103/PhysRevApplied.11.064044.
- [27] Mengyao Zhao, Bo Wu, and Jianji Dong. "On-chip multifunctional self-configurable quadrilateral MZI network". EN. In: *Optical Materials Express* 13.11 (Nov. 2023). Publisher: Optica Publishing Group, pp. 3138–3147. ISSN: 2159-3930. DOI: 10.1364/OME.499408.

- [28] William R Clements et al. "Optimal design for universal multiport interferometers: supplementary material". en. In: ().
- [29] Bryn A. Bell and Ian A. Walmsley. "Further Compactifying Linear Optical Unitaries". en. In: *APL Photonics* 6.7 (July 2021). arXiv:2104.07561 [quant-ph], p. 070804. issn: 2378-0967. doi: 10.1063/5.0053421.
- [30] J. Kennedy and R. Eberhart. "Particle swarm optimization". en. In: *Proceedings of ICNN'95 - International Conference on Neural Networks*. Vol. 4. Perth, WA, Australia: IEEE, 1995, pp. 1942–1948. isbn: 978-0-7803-2768-9. doi: 10.1109/ICNN.1995.488968.
- [31] Ian A. D. Williamson et al. "Reprogrammable Electro-Optic Nonlinear Activation Functions for Optical Neural Networks". en. In: *IEEE Journal of Selected Topics in Quantum Electronics* 26.1 (Jan. 2020). arXiv:1903.04579 [physics], pp. 1–12. issn: 1077-260X, 1558-4542. doi: 10.1109/JSTQE.2019.2930455.
- [32] Martin Arjovsky, Amar Shah, and Yoshua Bengio. *Unitary Evolution Recurrent Neural Networks*. en. arXiv:1511.06464 [cs, stat]. May 2016.
- [33] Ruiting Wang et al. "Multicore Photonic Complex-Valued Neural Network with Transformation Layer". en. In: *Photonics* 9.6 (May 2022), p. 384. issn: 2304-6732. doi: 10.3390/photonics9060384.